9 CAPABILITY DOMAINS · 6 VIEWS · 18 BRANCHES · 75 NODES

Agent 技术能力地图
决策级参考版

给传统软件工程师的一张可执行地图：从熟悉的契约、隔离、调度、检索、可观测性出发，理解 Agent 平台、Agentic 应用、Harness、模型系统与后训练之间的完整边界。全图只有一个能力图，用六个视图投影：分层、领域森林、生命周期、控制面、自治等级、角色。每个能力节点下挂实践与技术项，技术项单独登记、随时可换。

CORPUS SHA256 dac3b442…f79 VISIBILITY ≠ REQUIREMENT EXAMPLES ≠ UNIVERSAL THRESHOLDS

CAPABILITY STABLE / TECHNOLOGY VOLATILE

9 + 1	18	75	86	39
能力域（含 1 个横切）	能力分支	可追溯能力节点	登记技术项	一手来源入口

01 / SCOPE & READING GUIDE

这不是岗位分类学，而是工程责任地图

> **本图的定位：**把「证据、全景、行动」放在同一张图上。能力按 **9 个域 + 1 个横切带**组织；A / B / C 是同一份能力数据的**角色投影**，用 P1–P4 表达深度，而不是用归属表达重要性。Harness 作为横切镜头单独展开。真实岗位经常混合这些责任，尤其是 Harness、平台架构师与负责人岗。

先读这五条

01 · 评测是工程，不是刷榜

15/25 岗提及评测，但原文要的是任务集、badcase 归因、回归和线上闭环。能读懂一条失败轨迹，比会背一个榜单分数更值钱。

02 · Harness 是组织，不是包装

它把模型、工具、上下文、状态、执行环境和反馈组织成可交付行为。模型是外部常数；Harness 是多数应用团队真正能控制的变量。

03 · 平台交付制品，Agentic 交付行为

平台的成功单位是「第 N 个团队仍能稳定运行」；Agentic 的成功单位是「真实任务分布上的完成率与成本」。两者接口必须显式。

04 · 传统能力是入场券，不是差异化

契约、幂等、隔离、可观测、多租户仍然有效。真正稀缺的是非确定性调试、语义正确性判定和每成功任务成本。

05 · 模型系统与 RL 是相邻专业

推理服务、后训练与 Agentic RL 不应被塞进平台树，也不能从全景消失。理解接口与成本，按岗位选择深度。

使用方式

自查节点 → 对岗选树 → 做出制品。不要按「读完」验收，按「能拿出证据」验收。

按可用时间选择阅读路径

时间	读什么	读完应能回答
15 MIN	01 执行摘要 → 02 证据 → 03 全景 → 03·5 领域森林	这个岗位在市场里大致属于哪条树？它的核心失败面是什么？
60 MIN	03·7 Harness → 03·6 运行视图 → 目标轨道 A/B → 07 迁移矩阵	我现有能力能迁移到哪，缺的三件事是什么？
REFERENCE	用搜索框查节点；按 JD 编号、技术名或领域回溯	一个具体能力如何验收、对应哪些 JD、去哪里读一手资料？

A · Agent Platform Engineering

让 Agent 可运行、可隔离、可恢复、可治理、可度量。主责执行环境、任务运行时、协议网关、平台控制面、身份策略、可观测和评测基础设施。

B · Agentic AI Engineering

让 Agent 在目标任务上有效、可靠、经济、可迭代。主责任务建模、模型与策略选择、上下文与记忆、工具语义、质量科学、产品行为和反馈闭环。

C · Adjacent Specializations

模型服务与推理、后训练与 Agentic RL、产品及组织领导力。它们不是附属知识；在对应岗位中就是主战场，但不需要每位平台工程师都达到同等深度。

如何读每个节点

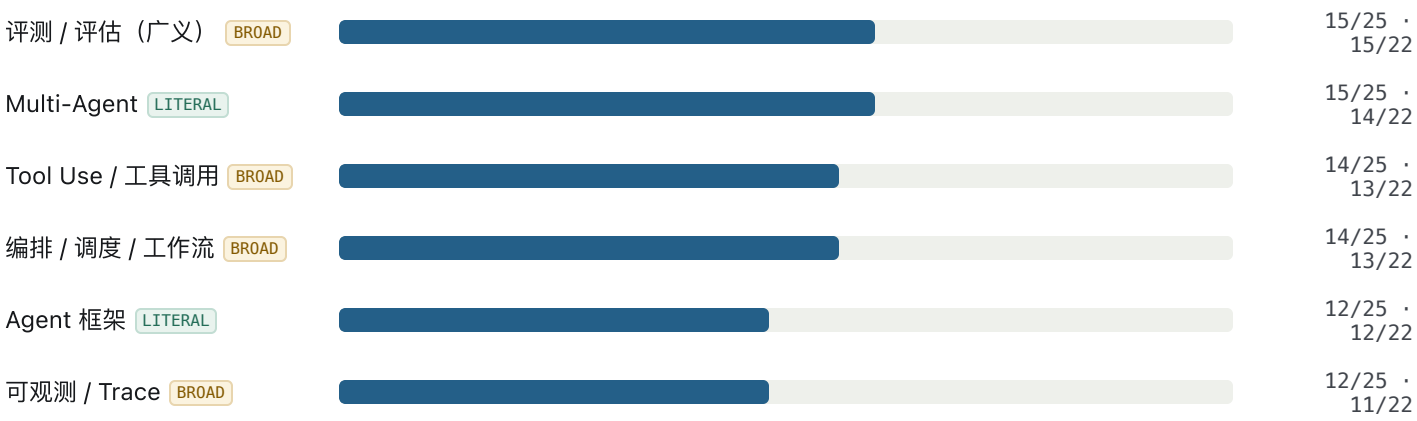
字段	含义
传统锚点	可以复用的既有工程心智；同时也会指出类比在哪里失效。
示例验收	用于学习或作品集的实验设计，不是跨行业发布阈值。样本量、风险等级、SLO 和统计置信度必须按业务校准。
JD	该节点在语料中的相关岗位编号，只表示相关证据，不代表每份 JD 都使用同一术语或把它列为硬性要求。
Sources	优先给规范、官方文档或官方工程文章。协议和框架快速变化，落地时应重新确认版本。

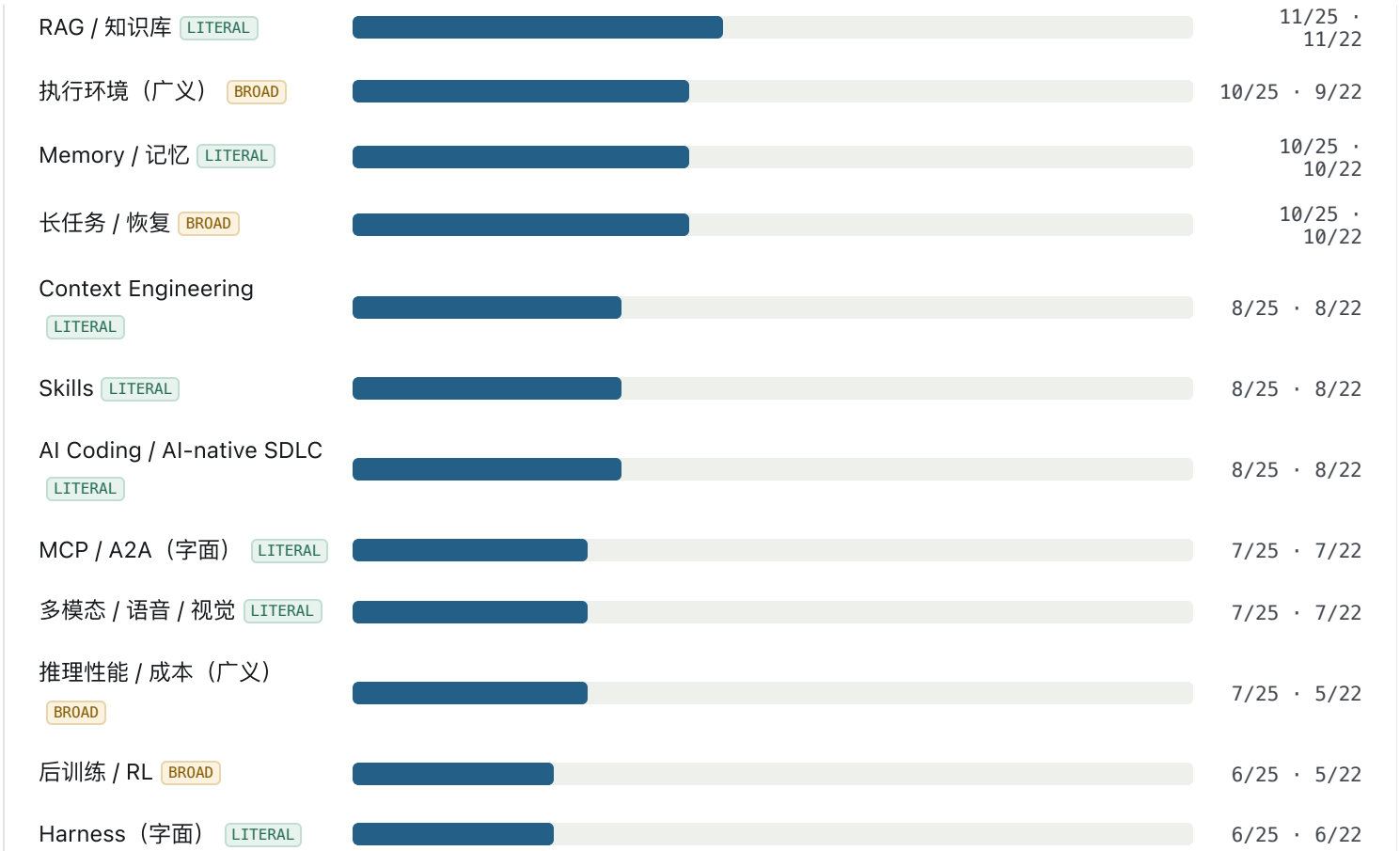
02 / EVIDENCE & CORRECTIONS

JD 可见度：只回答「出现了多少岗」

下表由随附的 `jd-capability-evidence.py` 重新生成。先清除组标题，再按岗位计数；核心视图排除 #19（产品）、#24（RL 研究）、#25（通用模型系统），仅用于敏感性对照，不代表这些方向不重要。

全部 25 岗　右侧同时显示核心 22 岗　BROAD 广义词族，需结合上下文　LITERAL 字面匹配





> **禁止从这张图直接推出：**硬性要求比例、市场份额、技能薪资溢价或学习优先级。它只说明这份便利样本中的术语可见度。岗位目标、业务风险和既有能力才决定学习顺序。

传统基础能力的字面核查

术语	岗位数	正确解释
Kubernetes	2 / 25	只在 #3、#11 明写；不能再用「K8s/云原生」混合词族替代字面计数。
微服务	3 / 25	#2、#3、#18。
中间件	5 / 25	#2、#3、#5、#9、#22。
Harness	6 / 25	#2、#6、#13、#14、#16、#17。不同 JD 中 Harness 语义不完全相同，不再做提及密度排名。

证据口径：这张图如何避免误读

常见误读	本图的处理

关键词出现多就是硬性要求	只称「JD 可见度」，不把提及升级为 required / core responsibility。
组标题、重复文本导致计数漂移	先清除组标题，按岗位计数；附可复现脚本与语料 SHA-256。
榜单分数等于生产能力	不引用不稳定榜单分数；验收按任务、风险和自有留出集校准。
把行业武断分成两种岗位	两条核心轨道 + 三个相邻专业，并增加按制品划分的责任接口。
模型系统、RL、AI Coding 不属于地图	作为相邻专业 C1/C2 与节点 B8 显式纳入；多模态进入 B7，领导力进入 C3。
广义词族与字面词混在一起比较	证据图每一行标注 `BROAD` 或 `LITERAL`，避免把两类数字直接比较。

03 / SYSTEM MAP & OWNERSHIP

全景不是严格分层，而是控制面相交

以下九层帮助定位能力，不代表请求只能逐层经过。安全、成本、评测和身份是横切关注点；模型服务与后训练可独立成平台。

L0	**算力与模型服务**	GPU/CPU、模型服务、KV cache、推理 SLO、模型与引擎版本	`C1 / A`
L1	**不受信执行环境**	沙箱、浏览器/计算机环境、镜像、文件系统、出口与凭证	`A`
L2	**工具与互操作**	Tool Schema、MCP、A2A、Skills、注册与版本、认证授权	`A + B`
L3	**持久任务运行时**	调度、状态、幂等、检查点、恢复、计时器、预算与容量	`A`
L4	**上下文与记忆**	上下文预算、压缩、检索、记忆写入/纠正/遗忘、租户隔离	`A + B`
L5	**Agent 决策与编排**	模型选择、ReAct/图/规划、委派、Multi-Agent、停止与人工接管	`B`
L6	**评测与反馈**	任务集、grader、轨迹、重复试验、线上信号、数据与反馈闭环	`A + B`
L7	**信任与经营**	身份、序列策略、隐私、发布、审计、每成功任务成本与事故响应	`A + B`
L8	**产品与组织**	领域流程、交互、多模态、ROI、采用、团队与治理	`B / C3`

同一张能力图的六个视图 + 一个横切镜头

视图不是并列的分类法，而是同一份能力数据的不同投影。节点编号在所有视图中保持一致，因此看到 A3 或 B6 时不需要换算。

视图	回答的问题	主键 / 单位	什么时候用	入口

① 分层	这项能力在系统哪一层，谁主责	运行时层 L0–L8	定位故障层级、划团队边界	本页
② 领域森林	它属于哪个能力域，下面还有几层	域 → 分支 → 能力 → 实践 → 技术项	决定学什么、检查覆盖是否完整	03·5
③ 生命周期	哪个阶段需要它，退出条件是什么	界定 → 构建 → 评测 → 发布 → 运行 → 学习 → 治理	排计划、做阶段评审	03·6
④ 控制面	它属于数据面还是控制面，原语是什么	平面 + 平台原语	平台架构、权限与成本设计	03·6
⑤ 自治等级	需要多强的监督与治理	R0–R4	风险分级、审批与接管设计	03·6
⑥ 角色	我这类岗位该深入到哪一层	A / B / C × P1–P4 深度	选岗、面试、团队梯队	04–06
✳ Harness	一次任务如何被组织成可交付行为	六个组件 + 失效面	调试、设计评审、归因	03·7

> **为什么不维护六套分类法：**六套并列的目录会立刻出现同名词条互相矛盾、更新不同步、读者无法定位。本图只维护一份能力数据（域 → 分支 → 能力），其余五个视图都是从这份数据派生出来的投影；技术项单独登记，因为它们的半衰期只有 6–18 个月。

按制品划分责任，而不是按岗位名称

制品 / 决策	主责	协作	接口与验收重点
沙箱与执行环境	`A`	B 定义任务需求；安全团队审查	威胁模型、隔离边界、镜像与依赖、出口、凭证、资源配额、销毁语义。
Agent 循环与停止策略	`B`	A 提供运行时与策略执行点	状态模型、最大步数/预算、暂停与恢复、人工接管、终止原因可观测。
工具契约	`A + B`	业务 API 与安全团队	B 负责可理解、可选择的语义；A 负责鉴权、版本、限流、审计和传输。
记忆模式与保留策略	`B`	A 提供存储、租户与合规能力	写入来源、正确性、用户纠正/删除、保留期、检索权限、记忆中毒测试。

评测任务与评分标准	B	领域专家、产品、A	真实任务分布、可判定结果、正例/反例、人工校准、任务集版本。
评测运行器与发布门禁	A	B 定义指标与风险门槛	环境隔离、模型/工具/配置固定、试验编排、结果存储、CI 集成、可审计比较。
模型服务与推理 SLO	C1	A 做接入治理；B 评估质量	TTFT、TPOT、吞吐、可用性、成本、模型/引擎版本和质量回归。
后训练与模型-Harness 协同	C2	B 提供任务与轨迹；A 提供数据/执行基建	数据血缘、奖励与环境、污染控制、离线与在线评测、安全回滚。
产品组合与自治边界	C3 + B	A/C1/C2 提供成本与能力约束	业务价值、风险等级、采用率、人工接管、退出机制、团队责任。

03·5 / CAPABILITY GRAPH

域是主键，角色只是投影

上一节的分层视图回答「能力在系统哪里」，本节回答「能力本身怎么组织」。这两件事需要分开：分层是运行时拓扑，会随架构演进；能力域是知识结构，稳定得多。全图只维护一份能力数据，角色轨道（A/B/C）是从它派生出来的阅读投影。

> **分类学结论：**A / B / C 保留，但降级为**角色投影**，不再作为根节点。根节点改为**能力域**：9 个域 + 1 个横切带。原因不是「两分法错」，而是它把三种不同的轴压在了同一层——岗位角色（A/B）、系统责任（分层）、专业成熟度（「相邻」暗含次要）。一旦需要新增方向，这种结构就只能在「再加一棵树」和「塞进现有树」之间二选一。

为什么保留 A/B/C，但不把它当根节点

问题	具体现象	本图处理
混轴	A / B 是岗位角色，C 是「相邻」，同一层级用了两种划分标准。	角色降为投影；根节点改为能力域，角色以深度列出现在覆盖矩阵中。
共管被拆散	工具契约（A3）与工具语义（B5）本来是同一条链，却被分到两棵树里，读者要来回跳。	领域视图把两者放回「互操作与契约」这一个域，共管关系直接可见。
深度被误读	「相邻」听起来次要，但 C1 模型服务、C2 后训练在对应岗位上就是主战场。	用 P1–P4 表达深度，而不是用归属表达重要性；同一分支对不同角色给不同目标等级。
技术绑在能力上	框架与协议名每年变化，写在能力节点里会让整棵树快速过期。	能力节点保持稳定；技术项进入独立登记册，标注类型与状态，可整体替换。

研究依据与本图的取舍

选型标准：能否把「稳定的能力」与「易变的实现」分开；能否容纳治理与生命周期；能否表达深度而不是只表达归属。

框架 / 来源	它提供什么	本图如何采用
CNCF 平台工程术语表	平台 = 能力 + 质量属性；具体实现是可替换的 provider。	**采用**：能力节点与技术项分离，技术登记册独立维护。
NIST AI RMF	GOVERN / MAP / MEASURE / MANAGE 四个功能，治理是贯穿功能而非末端阶段。	**采用**：治理做成横切带 D0，同时保留生命周期中的治理阶段。
ISO/IEC 42001	AI 管理体系要求，覆盖组织责任与系统生命周期。	**采用**：生命周期末端的治理与退出条件。
AWS · AgentOps 与 AgentCore 原语	Runtime、Memory、Gateway、Identity、Browser、Code Interpreter、Observability、Evaluations 等平台原语，以及 AgentOps 阶段。	**采用**：控制面视图的原语清单与生命周期阶段划分。
Anthropic · Building effective agents	工作流与 Agent 的区别，以及自治程度应按任务复杂度选择。	**采用**：R0–R4 自治等级视图。
SFIA 9	七级责任由通用属性（自主性、影响、复杂度、知识）定义，而不是由职级名称定义。	**借鉴**：压缩为 P1–P4 四档深度，不与 SFIA 七级一一映射。
NICE Framework	工作角色（work role）不等于职位名称；任务是任务、能力是能力。	**采用**：角色与能力分离，角色只用于投影。
Agentic AI Foundation	MCP、goose、AGENTS.md 作为创始项目进入中立治理（2025-12-09）；A2A 是 Linux Foundation 的独立项目。规范归属与厂商开始解耦。	**采用**：协议节点标注「规范状态」，而不是厂商功能状态。
ETCLOVG Harness 综述（OpenReview）	把 Harness 拆成七层的词汇体系。	**不采用为骨架**：该投稿在 OpenReview 上已撤回，只能作词汇参考，不能作为结构依据。

> **一个方法论上的取舍：**本图没有采用任何单一的「权威分类标准」，因为没有。平台工程、AI 治理、技能框架各自只覆盖一部分。可复用的是**结构原则**——稳定层与易变层分离、治理横切、深度而非归属——而不是某份清单本身。

五级节点模型：森林 → 树 → 枝 → 叶 → 叶脉

同一棵树上，越往下越具体、越易变、越需要一线判断。层级的存在意义是让你知道「该在哪一层做决策」。

层级	数量	半衰期	例子	维护者

L1 域 Domain · 森林	9 + 1 横切	5–10 年	执行与隔离、上下文与知识、模型系统	架构负责人
L2 分支 Branch · 树	18	3–5 年	不受信执行与环境（A1）	方向 owner
L3 能力 Capability · 枝	75	2–4 年	威胁模型与隔离分级	团队 tech lead
L4 实践 Practice · 叶	每节点 3–5 条	1–2 年	出口默认拒绝、凭证代理、按序列判定权限	一线工程师
L5 技术项 Technology · 叶脉	86 项	6–18 个月	Firecracker / gVisor / Kata / Temporal	平台或基础设施团队

> **为什么把技术项单独拿出来：**换掉叶脉不会让叶子消失，但会决定它还能不能活。技术项是可替换的导管——它支撑能力，却不应定义能力。把 Firecracker 写进能力节点的那一刻，这棵树就开始过期了。

深度刻度：同一分支，不同角色要求不同的深度

P1 · 认知

能解释术语与边界，能读懂他人的设计与产出，不独立决策。用来避免「不知道自己不知道」。

P2 · 会用

能在指导下完成实现，能验收常规产出；遇到无先例的问题知道该找谁升级。

P3 · 主责

能独立设计、交付与运维，能定义验收标准，能处理跨团队的接口与冲突。

P4 · 权威

能制定标准与取舍框架，能处理无先例问题，能培养 P3，并对组织级接口负责。

领域森林：9 个域 + 1 个横切带

每个域下列出所属分支与节点数。D0 是横切带：它不属于任何单一域，而是所有域的约束条件。

D0	**信任、安全与治理** TRUST & GOVERNANCE 横切所有域：权限在模型之外判定，风险随行动序列升级，而不是随单次调用升级。 A5 身份、策略、隐私与发布	4 NODES
D1	**执行与隔离** EXECUTION & ISOLATION 把模型生成的代码与动作当成不可信主体，并给长任务提供真正的恢复语义。 A1 不受信执行与环境	8 NODES

A2 持久任务运行时与调度

D2	**互操作与契约** INTEROP & CONTRACTS 同一套契约既要机器可执行，也要模型可理解；A 与 B 在这一域共管。 A3 工具、协议与 Skill 网关 B5 工具语义与 Skill 设计	9 NODES
D3	**上下文与知识** CONTEXT & KNOWLEDGE 决定下一步看到什么、长期记住什么、检索到什么，以及谁有权看到。 B3 上下文工程与压缩 B4 记忆、检索与知识治理	8 NODES
D4	**决策与编排** DECISION & ORCHESTRATION 任务是否适合 Agent、循环怎么走、何时停止、何时委派。 B1 任务-模型匹配与 Agent 设计 B2 规划、委派与 Multi-Agent	8 NODES
D5	**质量与评测** QUALITY & EVALUATION 任务集、评分器、重复试验与发布门禁；能力与统计是两件事。 A7 评测基础设施与发布门禁 B6 质量科学、统计与安全评测	10 NODES
D6	**运行、可观测与成本** OPERATIONS & ECONOMICS 以轨迹为调试单位，以成功任务为经营单位。 A6 轨迹可观测、成本与运行	4 NODES
D7	**平台控制面与开发体验** CONTROL PLANE & DX 把单点复杂度变成可复用、可迁移、可运维的平台制品。 A4 平台控制面与开发体验	4 NODES
D8	**模型系统** MODEL SYSTEMS 推理服务、后训练与 Harness 之间的能力与成本边界。 C1 模型服务与推理系统 C2 后训练与 Agentic RL	8 NODES

D9

产品、交互与组织

PRODUCT & ORGANIZATION

价值、信任、采用与责任由真实流程决定，不由 Agent 步数决定。

B7 领域产品、多模态与人机协作

B8 反馈学习与 AI-native 研发

C3 产品组合与组织领导

12 NODES

覆盖矩阵：同一分支在三个角色下的目标深度

读法：P4 是该角色的主战场，需要能定标准；P1 表示只需能对话与验收。矩阵不评价个人，只描述岗位期望。

分支	域	Platform	Agentic	Adjacent	节点
A1 不受信执行与环境	D1	P4	P2	P1	4
A2 持久任务运行时与调度	D1	P4	P2	P1	4
A3 工具、协议与 Skill 网关	D2	P4	P3	P1	5
A4 平台控制面与开发体验	D7	P4	P2	P1	4
A5 身份、策略、隐私与发布	D0	P3	P2	P2	4
A6 轨迹可观测、成本与运行	D6	P4	P3	P1	4
A7 评测基础设施与发布门禁	D5	P3	P3	P1	4
B1 任务-模型匹配与 Agent 设计	D4	P1	P4	P2	4
B2 规划、委派与 Multi-Agent	D4	P2	P4	P1	4
B3 上下文工程与压缩	D3	P2	P4	P2	4
B4 记忆、检索与知识治理	D3	P3	P4	P2	4
B5 工具语义与 Skill 设计	D2	P3	P4	P1	4
B6 质量科学、统计与安全评测	D5	P2	P4	P2	6
B7 领域产品、多模态与人机协作	D9	P1	P3	P3	4
B8 反馈学习与 AI-native 研发	D9	P2	P3	P2	4

分支	域	Platform	Agentic	Adjacent	节点
C1 模型服务与推理系统	D8	P2	P2	P4	4
C2 后训练与 Agentic RL	D8	P1	P2	P4	4
C3 产品组合与组织领导	D9	P1	P2	P4	4

03·6 / OPERATING VIEWS

运行视图：阶段、平面与自治等级

上面的视图回答「有哪些能力」。这一节回答三个运维问题：什么时候需要它、它在哪个平面执行、需要多强的治理。三个视图都从同一份能力数据派生，因此可以直接互相跳转。

生命周期视图：七个阶段与各自的退出条件

阶段不是瀑布。评测会回到构建，运行会回到评测；但每个阶段都必须有明确的退出条件，否则「做完了」只能靠感觉判断。

阶段	核心问题	主要节点	关键制品	退出条件
00 界定 Frame	这个任务该不该由 Agent 做，失败代价多大	B1 · C3	任务说明书、风险分级、明确的非目标	成功与失败都可判定，且非目标已写下来
01 构建 Build	循环、工具、上下文、状态是否闭合	B2 · B3 · B5 · A1 · A2	可运行纵切片、工具契约、trace	单任务端到端可复现，失败有可读轨迹
02 评测 Evaluate	它是否真的更好，好多少	B6 · A7	任务集、评分器、基线与反例报告	有基线、有反例、有重复试验与区间
03 发布 Release	变更能否安全放量	A5 · A7	门禁、灰度方案、回滚演练记录	关键失败路径有硬门禁，回滚被真实执行过
04 运行 Operate	现在是否健康，贵不贵	A2 · A6	trace/transcript、SLO、每成功任务成本台账	故障能定位到具体节点与轨迹，成本可归因
05 学习 Learn	同类失败如何不再重复	B8 · C2	badcase → 回归用例、数据与反馈闭环	同类失败率下降可测量，而不是靠复盘会
06 治理 Govern	谁负责，什么时候该退出	A5 · C3	RACI、审计记录、退出与降级方案	责任人与退出机制已签署，且有触发条件

控制面视图：数据面做事，控制面决定能做什么

这张表回答一个常见争议：为什么「加个网关」不能解决 Agent 的权限问题——因为权限判断需要行动序列，而序列只有控制面看得到。

平面	职责	关键原语	典型失效	节点
数据面 Data plane	真正执行模型推理、工具调用、检索与代码	推理引擎、沙箱、浏览器/计算机环境、向量检索、工具运行时	尾延迟、沙箱逃逸、工具超时、检索越权	C1 · A1 · A3 · B4
控制面 Control plane	决定允许做什么、给多少资源、用哪个版本	Agent 定义清单、策略引擎、调度与配额、注册与目录、发布与回滚	策略绕过、版本漂移、配额饥饿、孤儿制品	A4 · A5 · A2 · A3
管理面 Management plane	决定为什么做、值不值、谁负责	预算与 ROI、风险分级、供应商与退出成本、组织责任	无 owner、成本失控、退出无路、影子 Agent	C3 · B7

自治等级视图：按后果选自治，而不是按技术选自治

等级越高，能力要求与治理成本同时上升。多数生产系统应该停在 R2——不是因为技术不够，而是因为 R3 的恢复演练和值班成本往往超过收益。

等级	决策自由度	必须具备	治理要求	不该使用
R0 固定工作流	无模型决策，模型只做单步生成	确定性编排、结构化输出、校验	常规发布与抽检	输入分布稳定、规则可枚举时，加循环只会更差
R1 模型辅助	模型建议，人执行	输出契约、可解释性、错误回退	输出抽检与责任归属	需要多步工具调用才能完成任务
R2 有界 Agent	工具循环 + 预算 + 审批门	终止条件、幂等、审批、trace、回放	关键动作人工确认、失败告警	副作用不可逆且没有审批设计
R3 长任务自治	跨会话、可恢复、可接管	检查点、恢复演练、预算、人工接管、对账	值班、事故响应、定期演练	没有恢复演练、没有明确 owner
R4 多 Agent / 持续运行	委派、并行、长期记忆、自触发	协调成本测量、隔离、记忆治理、全局预算	组织级审计、退出机制、模型升级回归	单 Agent 尚未证明上下文或并行是瓶颈

> **一个反直觉的结论：**自治等级是**产品与风险决策**，不是技术成熟度奖励。把 R3 降回 R2 常常是工程进步：预算、审批和恢复语义带来的确定性，比「全自动」更有价值。

Harness：把模型能力组织成可交付行为

Harness 不是某个框架，也不是 Prompt wrapper，而是「从目标到最终环境状态」的运行时组织。它把模型、工具、上下文、状态、执行环境和反馈组织成可交付行为。

> **一句话：**模型决定「可能做什么」，Harness 决定「这次实际做什么、做完没有、失败怎么办」。对多数应用团队，模型是外部依赖，Harness 才是可控制、可评测、可迭代的工程对象。

01 · Loop & termination

目标、计划、行动、观察、停止条件、预算与人工接管。主责 B；运行时执行点由 A 提供。

`B` 制品：循环策略 + 终止原因 + 运行清单

02 · Model & prompt contract

模型选择、指令层级、结构化输出、模型路由与版本。主责 B；推理服务与容量由 C1 提供。

`B` 制品：模型/Prompt manifest + 输出契约

03 · Tools & skills

工具语义、Schema、Skill、MCP、A2A、注册、鉴权、版本与供应链。A 主契约与网关，B 主可用性。

`A + B` 制品：工具契约 + Skill 包 + 权限策略

04 · Context & memory

写、选、压、隔离；工作态、长期记忆、检索和用户纠正。A 主存储与租户，B 主写什么与选什么。

`A + B` 制品：上下文预算 + 记忆模式 + 删除路径

05 · Execution & recovery

沙箱、文件系统、浏览器、检查点、幂等、重试、暂停、恢复与销毁。

`A` 制品：隔离环境 + 恢复演练 + 副作用台账

06 · Eval & telemetry

任务集、轨迹、最终状态、评分器、重复试验、线上反馈和发布门禁。

`A + B` 制品：评测报告 + 可回放 trace + 回归门禁

Harness 的失效面与责任归属

失效面	典型症状	主责	先做哪个制品
循环不收敛	重复调用、预算耗尽、做完了还在做，或永远不结束。	`B2 / A2`	步数上限、循环检测、预算、终止原因与取消传播。
任务假成功	回答声称完成，但数据库、文件或外部系统没有正确最终状态。	`B6`	最终状态断言优先于文本评分；轨迹与 outcome 分开评。
恢复重复副作用	进程崩溃后重试，重复发信、扣款、写单或调用外部 API。	`A2`	幂等键、事务边界、outbox/inbox、记录回放与人工对账。
注入与数据外泄	网页、工具描述或检索文档中的指令改变目标；敏感数据被带出。	`A1 / A5`	威胁模型、默认拒绝出口、凭证代理、关键路径红队测试。
模型升级回归	换模型或 Prompt 后，行为、工具选择、成本或安全边界漂移。	`A5 / A7 / B6`	固定模型/Prompt/工具 manifest，灰度比较，过评测门禁再放量。

> **Harness is not a framework:** LangGraph、OpenAI Agents SDK、Claude Agent SDK、ADK 等是实现选择；Harness 的验收面是**可观察的任务行为**，不是你是否用了某个图、某个角色或某个中间件。先把行为与失败面写清楚，再选实现。

04 / ROLE PROJECTION · A

Agent Platform Engineering

平台不是把传统微服务换成 Agent 名称。新增难点是：模型参与控制流、执行不受信动作、轨迹成为调试对象、质量与成本共同进入发布门禁。

这是同一张能力图的角色投影：每个分支右上角标注所属领域（D0–D9）。同一分支在其他角色下的目标深度，见 03·5 覆盖矩阵。

A1 **不受信执行与环境** Untrusted execution

把模型生成的代码与浏览器动作当作不可预测、可能被注入的执行主体。

D1 · 执行环境 10/25
4 NODES

威胁模型与隔离分级

JD #3 #7 #11 #14

Threat-driven isolation

是什么 / WHY

按数据敏感度、代码来源、网络与副作用选择容器、用户态内核、microVM 或 VM；不存在脱离配置的绝对安全排序。

传统锚点 / ANCHOR

容器、VM、seccomp、威胁建模

示例验收 / EVIDENCE

为一个同时读取网页、私有仓库并执行代码的 Agent 写威胁模型，证明高风险路径没有共享凭证或不受限出口。

关键技术 / TECH

Firecracker / gVisor / Kata / seccomp

JD #3 #7 #11 #14 · Sources Firecracker ↗ gVisor ↗ Kata Containers (GitHub) ↗ OWASP AI Agent and MCP Security ↗

出口与凭证隔离

JD #11 #14 #21

Egress & credential containment

是什么 / WHY

注入无法被可靠根除，因此必须限制成功注入后的通信、凭证和副作用。

传统锚点 / ANCHOR

Egress firewall、secret broker、短期凭证

示例验收 / EVIDENCE

用网页诱导、工具描述投毒和数据外泄样本测试；同时报告绕过、误报、关键泄露路径和人工审批行为。

关键技术 / TECH

Egress allowlist / token vault / OIDC

JD #11 #14 #21 · Sources OWASP AI Agent and MCP Security ↗

环境快照与记录回放

JD #7 #11 #14

Snapshot & recorded replay

是什么 / WHY

环境快照只保存环境状态；可复核调试还需模型/工具 I/O、版本、配置和外部副作用记录。

传统锚点 / ANCHOR

VM snapshot、event sourcing、record/replay

示例验收 / EVIDENCE

分别演示状态恢复、基于记录的轨迹回放和实时重跑，并明确三者哪些结果可以比较、哪些不能。

关键技术 / TECH

Snapshot / event log / deterministic mocks

JD #7 #11 #14 · Sources Temporal Documentation ↗

浏览器与计算机环境

JD #11 #14

Browser / computer-use environment

是什么 / WHY

管理 DOM/截图输入、下载、文件系统、会话、外部账号和不可逆操作，不只是启动浏览器。

传统锚点 / ANCHOR

远程浏览器、VDI、自动化测试

示例验收 / EVIDENCE

在隔离环境复现下载替换、页面注入、会话过期和支付前审批；核查最终环境状态而非文本承诺。

关键技术 / TECH

Playwright / Browser Use / isolated VDI

JD #11 #14 · Sources OSWorld ↗ OWASP AI Agent and MCP Security ↗

A2

持久任务运行时与调度 Durable runtime & scheduling

把 Agent 运行建模为带副作用、预算和恢复语义的长生命周期工作流。

D1 · 长任务 10/25 · 编排 14/25
4 NODES

持久执行与幂等副作用

JD #4 #6 #9 #17

Durable execution & idempotency

是什么 / WHY

步骤可重试，但外部副作用不能简单重放；需要幂等键、事务边界、outbox/inbox 或补偿。

传统锚点 / ANCHOR

工作流引擎、消息队列、Saga

示例验收 / EVIDENCE

kill -9 后恢复同一任务，验证不会重复扣款、发信或写单；记录仍需人工对账的场景。

关键技术 / TECH

Temporal / DB queue / outbox / idempotency key

JD #4 #6 #9 #17 · Sources Temporal Documentation ↗

检查点、暂停与恢复

JD #4 #6 #11 #21

Checkpoint & recovery

是什么 / WHY

保存任务状态、待审批、计时器、工具游标和环境引用；模型输出缓存是另一项独立能力。

传统锚点 / ANCHOR

Checkpoint、持久化定时器、状态机

示例验收 / EVIDENCE

在模型调用、工具执行和人工审批前后分别故障注入，验证状态、预算和终止原因可恢复。

关键技术 / TECH

Checkpoint store / durable timer / state machine

JD #4 #6 #11 #21 · Sources Temporal Documentation ↗

任务调度与资源配额

JD #2 #3 #11 #18

Task & resource scheduling

是什么 / WHY

同时调度任务优先级和 CPU/GPU/沙箱容量，处理长任务、排队、公平性与取消。

传统锚点 / ANCHOR

K8s scheduler、队列、配额、抢占

示例验收 / EVIDENCE

构造混合长短任务与故障租户，观察尾延迟、配额公平、取消传播和恢复积压，而非只报吞吐。

关键技术 / TECH

Kubernetes / fair queue / priority / quota

JD #2 #3 #11 #18 · Sources Kubernetes Scheduling ↗

运行预算与终止控制

JD #4 #14 #17 #21

Budget & termination

是什么 / WHY

把步数、墙钟时间、token、工具费用和人工等待纳入可取消预算，避免无限循环。

传统锚点 / ANCHOR

超时、熔断、配额、kill switch

示例验收 / EVIDENCE

分别触发模型循环、工具重试风暴和外部系统不可用，验证预算耗尽后安全终止并可审计。

关键技术 / TECH

Budget controller / circuit breaker / cancellation

JD #4 #14 #17 #21 · Sources OWASP AI Agent and MCP Security ↗

A3

工具、协议与 Skill 网关 Tools, protocols & skills

连接层同时承担机器契约、模型可读语义、身份与供应链治理。

D2 · Tool 14/25 · MCP/A2A 7/25
5 NODES

工具契约与版本治理

JD #3 #16 #21

Tool contract & versioning

是什么 / WHY

统一 Schema、错误、超时、幂等、权限和兼容策略，让模型语义与机器执行可分别演进。

传统锚点 / ANCHOR

OpenAPI、gRPC、JSON Schema、版本化

示例验收 / EVIDENCE

跨版本运行契约测试；验证模糊参数、部分失败、限流和重复请求都有机器可读语义。

关键技术 / TECH

JSON Schema / OpenAPI / semantic versioning

JD #3 #16 #21 · Sources MCP 2026-07-28 release ↗

MCP 客户端与服务端

JD #3 #5 #12 #13 #16 #17 #21

MCP integration

是什么 / WHY

理解 Tools/Resources/Prompts、发现、传输、授权和扩展；按部署版本兼容，不假设所有实现同期升级。

传统锚点 / ANCHOR

RPC、OAuth、API gateway

示例验收 / EVIDENCE

在同网关后运行无状态多实例，验证路由、能力发现、列表缓存、授权和版本协商；记录不支持的新特性。

关键技术 / TECH

MCP 2026-07-28 / MRTR / CIMD / Tasks extension

JD #3 #5 #12 #13 #16 #17 #21 · Sources MCP 2026-07-28 release ↗

A2A 跨 Agent 互操作

JD #5 #21

A2A interoperability

是什么 / WHY

面向独立、 opaque Agent 的发现、任务与消息互操作，不是进程内 subagent 协议。

传统锚点 / ANCHOR

Agent Card、任务 API、远程过程调用

示例验收 / EVIDENCE

跨框架发现远程 Agent，验证 Agent Card 签名、任务轮询/流式、认证失败和不受信返回内容。

关键技术 / TECH

A2A v1.0 / JWS / JCS / Agent Card

JD #5 #21 · Sources A2A v1.0.0 Specification ↗

工具与Skill注册治理 Registry & supply chain

JD #3 #13 #20 #21

是什么 / WHY

对工具、MCP server、Skill 和脚本做来源、版本、权限、签名、审查和撤回，防止描述与依赖投毒。

传统锚点 / ANCHOR

Service catalog、artifact registry、SBOM

示例验收 / EVIDENCE

变更工具描述或 Skill 依赖后触发重新审查；演示撤回版本不会继续被新任务加载。

关键技术 / TECH

Registry / policy / SBOM / signed artifacts

JD #3 #13 #20 #21 · Sources Agent Skills Specification ↗ OWASP AI Agent and MCP Security ↗

协议网关与认证 Protocol gateway & auth

JD #3 #18 #21

是什么 / WHY

在模型之外执行认证、授权、限流、审计、路由和协议适配；模型不可决定自身权限。

传统锚点 / ANCHOR

API gateway、OPA、服务网格

示例验收 / EVIDENCE

验证不同租户、Agent 身份和工具版本的路由与权限；测试凭证跨域、重放和降级路径。

关键技术 / TECH

Gateway / OAuth / OPA / rate limit

JD #3 #18 #21 · Sources MCP 2026-07-28 release ↗ Open Policy Agent ↗

A4

平台控制面与开发体验 Control plane & developer experience

把单个 Agent 的复杂度变成可复用、可迁移、可运维的平台制品。

Agent 定义与配置模型

JD #2 #5 #9 #18

Agent definition & config

是什么 / WHY

将模型、指令、工具、Skills、运行时、权限、预算和评测配置组成可版本化制品。

传统锚点 / ANCHOR

CRD、配置即代码、IaC

示例验收 / EVIDENCE

同一 Agent 定义可部署到开发与生产并明确差异；任何变更都能关联评测和发布记录。

关键技术 / TECH

YAML/CRD / GitOps / immutable version

JD #2 #5 #9 #18 · Sources Amazon Bedrock AgentCore Harness GA ↗

框架适配与可移植性

JD #1 #3 #12 #16

Framework adapters

是什么 / WHY

隔离框架专有状态、事件和工具模型，避免把平台契约泄漏到每个业务 Agent。

传统锚点 / ANCHOR

Adapter、anti-corruption layer、SDK

示例验收 / EVIDENCE

用同一任务在两个框架实现纵切片，比较状态、追踪、恢复和工具语义，而非只比较代码行数。

关键技术 / TECH

LangGraph / OpenAI Agents SDK / ADK / Claude SDK

JD #1 #3 #12 #16 · Sources LangGraph Documentation ↗ OpenAI Agents SDK ↗ Google Agent Development Kit ↗ Claude Agent SDK ↗

环境、目录与自助发布

JD #2 #18 #20

Environment & catalog

是什么 / WHY

提供模板、环境、密钥引用、目录、审批和发布工作流，让团队不重复搭建基础设施。

传统锚点 / ANCHOR

Internal developer platform、Backstage

示例验收 / EVIDENCE

新团队从模板创建受治理 Agent，并自动获得身份、trace、评测和销毁策略。

关键技术 / TECH

IDP / templates / service catalog

JD #2 #18 #20 · Sources Amazon Bedrock AgentCore Harness GA ↗

测试 Harness 与环境虚拟化

JD #6 #13 #14 #20

Test harness & virtualization

是什么 / WHY

为 Agent 提供可控模型、工具、用户和时间替身，支持重复试验与故障注入。

传统锚点 / ANCHOR

Testcontainers、VCR、service virtualization

示例验收 / EVIDENCE

同一任务可在无真实外部副作用的环境重复运行，并覆盖超时、乱序、限流和异常工具返回。

关键技术 / TECH

Mock model / tool simulator / virtual user

JD #6 #13 #14 #20 · Sources Anthropic · Demystifying evals for AI agents ↗

A5

身份、策略、隐私与发布 Identity, policy & change

权限判断发生在模型之外，并考虑上下文、行动序列、数据生命周期与变更风险。

D0 · 安全治理横切
4 NODES

租户、用户与OBO身份

JD #3 #18 #21 #23

Tenant & OBO identity

是什么 / WHY

区分平台、租户、用户、Agent 与工具身份；跨 Agent 调用需保留委托链和受众边界。

传统锚点 / ANCHOR

IAM、OAuth、OIDC、ABAC/ReBAC

示例验收 / EVIDENCE

越权用例必须失败且可审计；跨 Agent 请求不能借复用高权限 token 扩大受众。

关键技术 / TECH

OAuth / OIDC / OBO / OpenFGA

JD #3 #18 #21 #23 · Sources OpenFGA Documentation ↗ A2A v1.0.0 Specification ↗

上下文与序列感知策略

JD #17 #21 #23

Context & sequence policy

是什么 / WHY

权限不仅看单次工具调用，还看此前读取过的数据、当前目标、序列与风险。

传统锚点 / ANCHOR

OPA、状态机、策略即代码

示例验收 / EVIDENCE

合法读取敏感数据后，后续外发动作被阻止或升级审批；策略判定不依赖模型自觉。

关键技术 / TECH

Policy engine / approval gate / risk tier

JD #17 #21 #23 · Sources Open Policy Agent ↗ OWASP AI Agent and MCP Security ↗

隐私与数据生命周期

JD #21 #23

Privacy & lifecycle

是什么 / WHY

对提示、轨迹、记忆、工具载荷和评测数据执行最小化、脱敏、保留、删除与驻留策略。

传统锚点 / ANCHOR

DLP、数据目录、保留策略

示例验收 / EVIDENCE

删除用户记忆后，在线索引、备份策略和评测副本按声明处理；trace 默认不采集敏感全文。

关键技术 / TECH

PII redaction / retention / tenant keys

JD #21 #23 · Sources OpenTelemetry GenAI Observability ↗ OWASP AI Agent and MCP Security ↗

版本、灰度与回滚

JD #2 #9 #18 #21

Release & rollback

是什么 / WHY

把模型、Prompt、工具 Schema、Skill、记忆策略和 Harness 作为联合变更表面。

传统锚点 / ANCHOR

GitOps、灰度、feature flag、回滚

示例验收 / EVIDENCE

一次模型或工具变更可定位到质量差异；回滚代码时不会遗留不兼容记忆或任务状态。

关键技术 / TECH

Immutable release / eval gate / canary

JD #2 #9 #18 #21 · Sources Anthropic · Demystifying evals for AI agents ↗

A6

轨迹可观测、成本与运行 Observability, cost & operations

调试单位是完整任务轨迹，经营单位是成功任务，而不是单次模型请求。

D6 · 可观测 12/25
4 NODES

端到端 Agent Trace Agent

JD #3 #4 #16 #21 #22

tracing

是什么 / WHY

串联任务、Agent、模型、工具、检索、审批、环境事件和最终结果，并记录配置版本。

传统锚点 / ANCHOR

OpenTelemetry、分布式追踪

示例验收 / EVIDENCE

任意失败任务可回答模型、工具、环境、上下文和策略分别发生了什么；敏感内容按需采集。

关键技术 / TECH

OTel GenAI / invoke_agent / execute_tool

JD #3 #4 #16 #21 #22 · Sources OpenTelemetry GenAI Observability ↗ OTel semantic-conventions-genai ↗

轨迹调试与失败归因

JD #11 #14 #16 #21

Trajectory

debugging

是什么 / WHY

建立可读轨迹、失败分类、环境状态和最终 outcome 的联合视图，避免只看最后回答。

传统锚点 / ANCHOR

日志分析、分布式调试、事件溯源

示例验收 / EVIDENCE

对一批失败做盲归因并与事后根因对照，验证分类体系能指导修复而非只统计数量。

关键技术 / TECH

Trace viewer / failure taxonomy / outcome check

JD #11 #14 #16 #21 · Sources Anthropic · Demystifying evals for AI agents ↗

每成功任务成本 Cost per successful task

JD #3 #8 #14 #17 #21

是什么 / WHY

成本由轨迹、重试、工具、沙箱、存储、网络、人工复核与失败共同决定；不是简单随时长线性。

传统锚点 / ANCHOR

FinOps、单位经济、成本分摊

示例验收 / EVIDENCE

按任务分位和成功/失败分层报告成本，识别重试、上下文与工具异常；同时观察 QPS 和并发影响。

关键技术 / TECH

Token metering / runtime cost / human review cost

JD #3 #8 #14 #17 #21 · Sources Anthropic · Multi-agent research system ↗

质量-延迟-可用性 SLO Multi-dimensional SLO

JD #3 #4 #21 #22

是什么 / WHY

任务成功、延迟、可用性、安全和成本都是生产目标，不能相互替代。

传统锚点 / ANCHOR

SRE、SLI/SLO、错误预算

示例验收 / EVIDENCE

分别定义短交互与长任务 SLO；故障演练同时观察成功率、尾延迟、恢复时间和副作用。

关键技术 / TECH

Task success / TTFT / E2E latency / safety budget

JD #3 #4 #21 #22 · Sources OpenTelemetry GenAI Observability ↗

A7 评测基础设施与发布门禁 Evaluation infrastructure

A 负责可重复执行和证据保存，B 负责任务与评分是否真正代表业务。

D5 · 评测可见度 15/25
4 NODES

任务与数据版本仓库

JD #13 #15 #20 #23

Task & dataset registry

是什么 / WHY

管理任务、输入环境、预期 outcome、风险标签、来源、许可和版本，防止评测集静默变化。

传统锚点 / ANCHOR

数据集版本、特征仓库、测试管理

示例验收 / EVIDENCE

每次报告都能定位任务集与环境版本；新增真实 badcase 不覆盖历史基准。

关键技术 / TECH

DVC / dataset registry / task manifest

JD #13 #15 #20 #23 · Sources Anthropic · Demystifying evals for AI agents ↗

隔离评测运行器

JD #6 #13 #14 #20

Evaluation runner

是什么 / WHY

并发运行任务与多 trials，固定模型、Harness、工具、环境和评分器配置。

传统锚点 / ANCHOR

CI、分布式任务队列、容器

示例验收 / EVIDENCE

同一配置可重复执行并保存原始轨迹；运行间资源与外部依赖差异被显式记录。

关键技术 / TECH

Eval harness / isolated workers / run manifest

JD #6 #13 #14 #20 · Sources Anthropic · Demystifying evals for AI agents ↗

评分器服务与校准数据

JD #13 #15 #17 #20

Grader service & calibration

是什么 / WHY

统一代码、模型和人工评分接口，保存 rubric、judge 版本与校准样本。

传统锚点 / ANCHOR

模型服务、标注平台、统计评估

示例验收 / EVIDENCE

Judge 更新可在固定人工集上比较一致性、偏差与成本，不会静默改变历史分数。

关键技术 / TECH

LLM-as-judge / human review / rubric registry

JD #13 #15 #17 #20 · Sources Anthropic · Demystifying evals for AI agents ↗

回归与发布证据

JD #14 #16 #17 #21

Regression evidence

是什么 / WHY

将质量、一致性、成本、延迟与安全结果纳入可审计比较；阈值由风险决定。

传统锚点 / ANCHOR

CI quality gate、实验平台、统计检验

示例验收 / EVIDENCE

报告样本量、每任务 trials、置信区间、版本和失败清单；小样本只作为探索性证据。

关键技术 / TECH

Paired comparison / CI / release gate

JD #14 #16 #17 #21 · Sources Anthropic · Demystifying evals for AI agents ↗

Agentic AI Engineering

Agentic 工程不是 Prompt 编写。核心是把开放任务转化为可约束的决策系统，并用轨迹、实验和产品信号持续证明系统是否变好。

两个相邻专业（C1 模型服务、C2 后训练）在这一轨道是 P2 深度：需要能定义接口与验收，但不需要自己训练模型。

B1

任务-模型匹配与 Agent 设计 Task-model fit & agent design

先决定任务是否适合 Agent，再决定模型、循环和确定性工作流的边界。

D4 · Agent 框架 12/25
4 NODES

任务可判定性与自治分级 Task legibility & autonomy

JD #1 #8 #10 #19

是什么 / WHY

评估目标是否可判定、环境是否可观察、错误是否可逆，再决定自动化程度。

传统锚点 / ANCHOR

工作流分析、风险分级、HITL

示例验收 / EVIDENCE

为一个任务写出适合 Agent、适合固定工作流和不应自动化三类子任务，并给出可验证理由。

关键技术 / TECH

Task decomposition / risk tier / autonomy level

JD #1 #8 #10 #19 · Sources Anthropic · Demystifying evals for AI agents ↗

模型与策略选择

JD #3 #9 #12 #20

Model & policy selection

是什么 / WHY

按任务质量、工具能力、上下文、延迟、成本和可用性选择模型与运行参数。

传统锚点 / ANCHOR

模型评测、A/B、路由

示例验收 / EVIDENCE

在自有任务上比较候选模型，固定 Harness 并重复试验；报告质量-成本前沿而非单次印象。

关键技术 / TECH

Model routing / structured output / decoding config

JD #3 #9 #12 #20 · Sources Anthropic · Demystifying evals for AI agents ↗

Agent 与确定性工作流边界

JD #2 #5 #9 #18

Agent/workflow boundary

是什么 / WHY

把开放决策交给模型，把稳定规则、资金、权限和关键校验交给确定性代码。

传统锚点 / ANCHOR

状态机、业务规则引擎

示例验收 / EVIDENCE

标出循环中每个决策由模型还是代码负责；高风险终态不能仅靠自然语言约束。

关键技术 / TECH

ReAct / state graph / rules / guardrails

JD #2 #5 #9 #18 · Sources LangGraph Documentation ↗

结构化输出与降级 Structured behavior & fallback

JD #17 #21 #23

是什么 / WHY

让输出可校验，并为模型不可用、低置信、工具失败和预算耗尽设计替代路径。

传统锚点 / ANCHOR

Schema validation、重试、降级

示例验收 / EVIDENCE

无效 JSON、拒答、工具超时和低置信分别进入明确路径，不会无限重试或静默成功。

关键技术 / TECH

JSON Schema / constrained decoding / fallback

JD #17 #21 #23 · Sources Anthropic · Demystifying evals for AI agents ↗

B2

规划、委派与 Multi-Agent Planning & delegation

多 Agent 是用协调成本换并行、隔离或专门化，不是能力树的默认终点。

D4 · Multi-Agent 15/25
4 NODES

计划、执行与再规划

JD #1 #4 #14 #17

Plan-execute-replan

是什么 / WHY

管理目标、中间假设、计划状态和计划失效，避免把固定步骤误称自主规划。

传统锚点 / ANCHOR

规划器、状态图、反思

示例验收 / EVIDENCE

在外部信息推翻初始计划时，系统能保留证据、调整计划并说明终止条件。

关键技术 / TECH

Plan-and-Execute / state graph / critique

JD #1 #4 #14 #17 · Sources LangGraph Documentation ↗

Subagent 与上下文隔离

JD #4 #13 #14

Subagent isolation

是什么 / WHY

把独立调查、不同工具权限或超出单上下文的工作拆分，并限制返回信息量。

传统锚点 / ANCHOR

并行任务、上下文工程

示例验收 / EVIDENCE

对比单 Agent 与 Subagent 的质量、token、延迟和失败传播；证明拆分来自任务结构。

关键技术 / TECH

Orchestrator-worker / scoped context

JD #4 #13 #14 · Sources Anthropic · Multi-agent research system ↗ Anthropic · Effective context engineering ↗

Multi-Agent 协作协议

JD #2 #8 #13 #20

Multi-agent coordination

是什么 / WHY

定义任务所有权、共享状态、消息语义、冲突、合并与取消；避免角色扮演式空拆。

传统锚点 / ANCHOR

黑板、消息传递、 supervisor

示例验收 / EVIDENCE

在重复工作、冲突结论和部分失败下验证合并与终止；与单 Agent 基线做同任务比较。

关键技术 / TECH

Supervisor / blackboard / A2A / handoff

JD #2 #8 #13 #20 · Sources Anthropic · Multi-agent research system ↗ A2A v1.0.0 Specification ↗

自治预算与人工升级

JD #4 #10 #17 #21

Autonomy & escalation

是什么 / WHY

根据风险、不确定性和连续失败升级人工，而不是仅按固定步数停止。

传统锚点 / ANCHOR

人机协作、异常队列、风险控制

示例验收 / EVIDENCE

构造目标含糊、工具冲突和高风险动作，验证 Agent 能提问、暂停、转交并保留上下文。

关键技术 / TECH

HITL / confidence policy / escalation queue

JD #4 #10 #17 #21 · Sources Anthropic · Demystifying evals for AI agents ↗ OWASP AI Agent and MCP Security ↗

B3 上下文工程与压缩 Context engineering

选择下一步决策所需的信息，而不是把最大上下文窗口当作默认存储。

D3 · Context 8/25
4 NODES

上下文预算与信息来源

JD #4 #9 #14 #17

Context budget & provenance

是什么 / WHY

管理指令、历史、工具结果、检索、记忆和 Skills 的优先级、来源与信任边界。

传统锚点 / ANCHOR

缓存、信息架构、优先级队列

示例验收 / EVIDENCE

报告各类上下文占比、来源与任务贡献；不受信内容与系统指令在策略上明确区分。

关键技术 / TECH

Context budget / provenance / prompt cache

JD #4 #9 #14 #17 · Sources Anthropic · Effective context engineering ↗ OWASP AI Agent and MCP Security ↗

压缩、摘要与状态外化

JD #13 #14 #16 #17

Compaction & external state

是什么 / WHY

压缩会丢失信息；关键状态应结构化外化，摘要需要保留未决事项、证据与来源。

传统锚点 / ANCHOR

摘要、状态外化、事件日志

示例验收 / EVIDENCE

长任务中比较全文、滚动摘要和结构化状态，测量质量、成本及关键事实丢失，不使用固定 60–70% 阈值。

关键技术 / TECH

Compaction / structured scratchpad / handoff state

JD #13 #14 #16 #17 · Sources Anthropic · Effective context engineering ↗

检索式上下文与按需加载

JD #3 #13 #16 #21

Just-in-time context

是什么 / WHY

按需读取工具、文件、记忆和 Skill，减少静态上下文，同时控制检索失败。

传统锚点 / ANCHOR

RAG、工具检索、渐进披露

示例验收 / EVIDENCE

大规模工具/Skill 集合下评估选择错误、遗漏、加载延迟和上下文成本。

关键技术 / TECH

Tool retrieval / SKILL.md / references

JD #3 #13 #16 #21 · Sources Agent Skills Specification ↗ Anthropic · Effective context engineering ↗

跨步骤状态与交接

JD #4 #13 #17

State & handoff design

是什么 / WHY

明确哪些状态属于任务、步骤、Agent 和用户，避免通过重复全文传递所有信息。

传统锚点 / ANCHOR

状态机、DTO、工作流上下文

示例验收 / EVIDENCE

跨 Agent/人工交接后，目标、证据、权限、未决事项和副作用状态仍可验证。

关键技术 / TECH

Handoff schema / task state / event log

JD #4 #13 #17 · Sources Anthropic · Effective context engineering ↗

B4

记忆、检索与知识治理 Memory & knowledge

Memory 是学习什么值得保留；RAG 是检索当前需要什么。两者都必须处理权限、时间和错误。

D3 · Memory 10/25 · RAG 11/25
4 NODES

记忆写入与来源治理

JD #5 #13 #17 #21

Memory write & provenance

是什么 / WHY

决定从对话、行为和反馈中写什么，并保存来源、置信、时间、租户和用户控制。

传统锚点 / ANCHOR

CDC、数据质量、元数据

示例验收 / EVIDENCE

错误推断、过期偏好和恶意写入可以被纠正、撤回和审计，不能永久污染后续任务。

关键技术 / TECH

Episodic/semantic memory / provenance / TTL

JD #5 #13 #17 #21 · Sources OWASP AI Agent and MCP Security ↗

记忆检索与反馈闭环

JD #3 #13 #17 #21

Memory retrieval & feedback

是什么 / WHY

按任务、时间、主体和权限检索，评估记忆是否有帮助而非仅召回相似文本。

传统锚点 / ANCHOR

检索排序、在线学习、反馈

示例验收 / EVIDENCE

比较无记忆与有记忆策略，并专门测试错误记忆、冲突记忆和遗忘后的行为。

关键技术 / TECH

Hybrid retrieval / reranking / feedback

JD #3 #13 #17 #21 · Sources Anthropic · Effective context engineering ↗

RAG 与知识状态校验

JD #8 #15 #20 #23

RAG & grounded outcome

是什么 / WHY

处理切分、索引、权限、引用、时效与答案后的环境状态，不把检索相似度当任务成功。

传统锚点 / ANCHOR

搜索、推荐、知识图谱

示例验收 / EVIDENCE

建立无答案、冲突来源、过期文档和越权文档用例，检查引用与最终状态。

关键技术 / TECH

Vector/BM25 / rerank / ACL-aware retrieval

JD #8 #15 #20 #23 · Sources Anthropic · Demystifying evals for AI agents ↗

纠正、遗忘与删除 Correction & forgetting

JD #21 #23

是什么 / WHY

用户纠正与删除需要穿透记忆、索引、缓存和派生数据，不只是追加一条新偏好。

传统锚点 / ANCHOR

数据生命周期、撤回、索引更新

示例验收 / EVIDENCE

执行纠正和删除后，新任务不再使用旧事实；保留策略和备份例外有明确记录。

关键技术 / TECH

Tombstone / reindex / retention policy

JD #21 #23 · Sources OWASP AI Agent and MCP Security ↗

B5 工具语义与 Skill 设计 Agent-facing tools & skills

把传统 API 转成模型能理解、选择、纠错和安全使用的能力接口。

D2 · Tool 14/25 · Skills 8/25
4 NODES

Agent-facing 工具设计 Agent-facing tool design

JD #1 #3 #14 #16

是什么 / WHY

工具名称、描述、参数和错误是模型决策界面；需要机器契约与语义契约同时成立。

传统锚点 / ANCHOR

API design、UX writing、Schema

示例验收 / EVIDENCE

在歧义任务和异常返回下评估选择、参数和恢复；模型之外的权限始终独立执行。

关键技术 / TECH

JSON Schema / error taxonomy / idempotency

JD #1 #3 #14 #16 · Sources Anthropic · Demystifying evals for AI agents ↗ MCP 2026-07-28 release ↗

Skill 包与渐进披露

JD #3 #5 #13 #21

Skill design & progressive disclosure

是什么 / WHY

Skill 外化程序性知识，可含指令、脚本、引用和资产；不等于 Tool，也不天然可信。

传统锚点 / ANCHOR

文档工程、代码包、版本管理

示例验收 / EVIDENCE

验证触发、误触发、正文长度、引用加载和脚本权限；官方建议元数据约 100 token、正文 <5000 token。

关键技术 / TECH

SKILL.md / scripts / references / skills-ref

JD #3 #5 #13 #21 · Sources Agent Skills Specification ↗ OWASP AI Agent and MCP Security ↗

大规模工具与 Skill 检索

JD #3 #13 #20 #21

Capability retrieval

是什么 / WHY

能力数量增长后需要检索、分组、权限过滤和上下文预算，不能全部塞入提示。

传统锚点 / ANCHOR

检索、路由、服务目录

示例验收 / EVIDENCE

随着能力数量增长报告选择准确率、遗漏、延迟和 token；按租户隐藏无权限能力。

关键技术 / TECH

Tool retrieval / registry / ranking

JD #3 #13 #20 #21 · Sources Agent Skills Specification ↗

工具结果压缩与可恢复错误

JD #14 #16 #17

Result shaping & recoverable errors

是什么 / WHY

把大型、噪声或敏感工具结果转成有来源、可继续行动的信息，并区分可重试与需改参。

传统锚点 / ANCHOR

错误处理、分页、摘要

示例验收 / EVIDENCE

工具超时、空结果、部分失败和大 payload 分别产生可处理行为，不造成盲目重复调用。

关键技术 / TECH

Pagination / typed errors / result summary

JD #14 #16 #17 · Sources Anthropic · Demystifying evals for AI agents ↗

B6

质量科学、统计与安全评测 Evaluation science & safety

同时回答平均能力、重复一致性、风险、成本与线上价值，避免单一分数。

D5 · 评测可见度 15/25
6 NODES

任务采样与成功定义

JD #13 #15 #20 #23

Task sampling & outcome

是什么 / WHY

从真实分布和失败中采样，定义可观察 outcome、交互质量、正例与反例。

传统锚点 / ANCHOR

测试设计、领域建模、数据采样

示例验收 / EVIDENCE

领域专家能独立判断任务是否成功；任务集覆盖常见、困难、边界和不应执行的场景。

关键技术 / TECH

Task set / gold outcome / negative cases

JD #13 #15 #20 #23 · Sources Anthropic · Demystifying evals for AI agents ↗

轨迹与结果联合评分

JD #14 #16 #17 #21

Trajectory & outcome grading

是什么 / WHY

代码校验、状态检查、rubric 与人工评审各评不同维度；有效替代路径不应被机械判错。

传统锚点 / ANCHOR

单元测试、状态断言、LLM Judge

示例验收 / EVIDENCE

最终回答声称成功时检查真实环境；阅读误判轨迹并修正任务或评分器。

关键技术 / TECH

Code grader / state grader / rubric / transcript

JD #14 #16 #17 #21 · Sources Anthropic · Demystifying evals for AI agents ↗

Judge 校准与偏差控制

JD #13 #15 #17 #20

Judge calibration

是什么 / WHY

Judge 是带版本和偏差的测量仪器，需要人工锚点、偏差测试和持续抽检。

传统锚点 / ANCHOR

标注、测量一致性、统计质量控制

示例验收 / EVIDENCE

固定人工集上报告一致性、系统性偏好和分歧案例；Judge 更新重新校准。

关键技术 / TECH

Human calibration / rubric / judge registry

JD #13 #15 #17 #20 · Sources Anthropic · Demystifying evals for AI agents ↗

重复试验与不确定度 Trials & uncertainty

JD #14 #16 #20

是什么 / WHY

pass@1 衡量单次平均成功，pass@k 衡量至少一次成功，pass^k 衡量全部成功；按产品问题组合使用。

传统锚点 / ANCHOR

实验设计、二项统计、A/B

示例验收 / EVIDENCE

固定配置进行多 trials，报告任务数、试验数、置信区间与配对差异；不把五次运行笼统称为稳定。

关键技术 / TECH

pass@1 / pass@k / pass^k / paired CI

JD #14 #16 #20 · Sources Anthropic · Demystifying evals for AI agents ↗

安全、滥用与红队 Safety & adversarial eval

JD #11 #17 #21 #23

是什么 / WHY

测试目标劫持、工具滥用、数据外泄、记忆中毒、跨 Agent 欺骗和人工信任滥用。

传统锚点 / ANCHOR

AppSec、威胁建模、红队

示例验收 / EVIDENCE

按风险定义关键不可接受结果；报告攻击覆盖、严重度、误报与防御深度，而非只报一个拦截率。

关键技术 / TECH

OWASP ASI / injection corpus / abuse cases

JD #11 #17 #21 #23 · Sources OWASP AI Agent and MCP Security ↗

线上反馈与回归资产

JD #16 #17 #20 #21

Online signals & regression

是什么 / WHY

把人工接管、纠正、重做、投诉、最终业务 outcome 和 badcase 回流为评测资产。

传统锚点 / ANCHOR

产品分析、闭环质量、A/B

示例验收 / EVIDENCE

线上问题能追溯到任务、轨迹、配置和修复回归；同时防止只优化易采集代理指标。

关键技术 / TECH

Online eval / user feedback / badcase loop

JD #16 #17 #20 #21 · Sources Anthropic · Demystifying evals for AI agents ↗

B7

领域产品、多模态与人机协作 Product, domain & interaction

Agent 的价值由真实流程、用户信任和组织采用决定，不由 Agent 步数决定。

D9 · 多模态 7/25
4 NODES

领域流程与业务状态建模

JD #8 #10 #19 #23

Domain

workflow

modeling

是什么 / WHY

把领域规则、角色、状态、异常、合规和最终业务结果显式建模。

传统锚点 / ANCHOR

DDD、BPMN、业务分析

示例验收 / EVIDENCE

领域专家可从轨迹判断业务正确性；系统处理异常与反向流程，而非只跑通 happy path。

关键技术 / TECH

Domain model / state machine / business outcome

JD #8 #10 #19 #23 · Sources Anthropic · Demystifying evals for AI agents ↗

人工接管与信任交互

JD #4 #10 #21

HITL & trust UX

是什么 / WHY

呈现目标、证据、风险、待批准动作和恢复状态，避免把审批做成无信息按钮。

传统锚点 / ANCHOR

UX、审批流、可解释界面

示例验收 / EVIDENCE

用户能发现并纠正错误计划、拒绝具体动作、接管后继续任务；记录接管原因。

关键技术 / TECH

Approval UX / progress / undo / escalation

JD #4 #10 #21 · Sources OWASP AI Agent and MCP Security ↗

多模态与实时交互

JD #1 #8 #10 #12

Multimodal interaction

是什么 / WHY

处理语音、图像、屏幕和文档输入的时延、成本、隐私、不确定性与模态回退。

传统锚点 / ANCHOR

实时音视频、OCR、多模态评测

示例验收 / EVIDENCE

在噪声、模糊图像和中断下测试任务结果与人工接管，而非只演示理想输入。

关键技术 / TECH

Speech / vision / realtime / OCR

JD #1 #8 #10 #12 · Sources Anthropic · Demystifying evals for AI agents ↗

价值、采用与退出机制 JD #8 #10 #19 #20

Value, adoption & exit

是什么 / WHY

衡量节省的时间、质量提升、风险、人工复核与用户采用，并保留非 Agent 退出路径。

传统锚点 / ANCHOR

产品指标、服务设计、FinOps

示例验收 / EVIDENCE

报告每成功任务总成本与用户/业务 outcome；低价值或高风险子流程可退回固定工作流。

关键技术 / TECH

ROI / adoption / human review / fallback

JD #8 #10 #19 #20 · Sources Anthropic · Multi-agent research system ↗

B8

反馈学习与 AI-native 研发 Learning loops & AI-native SDLC

自进化首先是受治理的工程闭环；AI Coding 也是可评审、可测试的研发体系。

D9 · AI Coding 8/25
4 NODES

反馈到待办与知识沉淀

JD #1 #16 #17 #20

Feedback-to-backlog

是什么 / WHY

把 badcase 归因到模型、上下文、工具、环境、策略或任务定义，再进入修复与回归。

传统锚点 / ANCHOR

质量闭环、根因分析、知识管理

示例验收 / EVIDENCE

每类系统性失败有负责人、修复假设、回归任务和验证结果，而非只追加提示补丁。

关键技术 / TECH

Failure taxonomy / eval-driven backlog

JD #1 #16 #17 #20 · Sources Anthropic · Demystifying evals for AI agents ↗

Skill 与经验学习

JD #1 #13 #16

Skill & experience learning

是什么 / WHY

从成功/失败轨迹提炼可复用流程，但必须经过审查、版本化、权限和效果验证。

传统锚点 / ANCHOR

知识蒸馏、规则挖掘、资产化

示例验收 / EVIDENCE

候选 Skill 在保留任务集上证明净收益，误触发、冲突和撤回路径均被测试。

关键技术 / TECH

Skill mining / trajectory mining / registry

JD #1 #13 #16 · Sources Agent Skills Specification ↗ Anthropic · Demystifying evals for AI agents ↗

有边界的自改进

JD #1 #13 #16 #17

Bounded self-improvement

是什么 / WHY

自动修改 Prompt、工具或策略需要离线验证、审批、灰度和回滚；不能无边界自我扩权。

传统锚点 / ANCHOR

AutoML、配置优化、变更管理

示例验收 / EVIDENCE

系统提出改进候选但不直接获得生产权限；所有变更可定位、可复现、可撤回。

关键技术 / TECH

Prompt/config optimization / gated rollout

JD #1 #13 #16 #17 · Sources Anthropic · Demystifying evals for AI agents ↗ OWASP AI Agent and MCP Security ↗

Spec Coding 与 AI 协作研发

JD #2 #3 #7 #12 #13 #14 #17 #20

Spec-driven AI coding

是什么 / WHY

把需求、架构、测试、代码审查和证据变成 Agent 可执行且人可评审的工程制品。

传统锚点 / ANCHOR

SDLC、规格驱动开发、代码评审

示例验收 / EVIDENCE

AI 生成变更必须通过规格、测试、安全审查与人工责任；评估返工和缺陷，而非只统计生成速度。

关键技术 / TECH

Spec Coding / AGENTS.md / eval-driven development

JD #2 #3 #7 #12 #13 #14 #17 #20 · Sources Anthropic · Demystifying evals for AI agents ↗ OWASP AI Agent and MCP Security ↗

相邻专业：全景中不可缺失的三块

这些方向可作为独立职业路径。平台或 Agentic 工程师至少需要理解其接口、成本与质量约束，但不必全部深入实现。

「相邻」描述的是与 A/B 的接口距离，不是重要性。在 C1、C2、C3 各自的岗位上，对应分支就是 P4 主战场。

C1

模型服务与推理系统 Model serving & inference

独立专业方向：为 Agent 提供可扩展、可测量、可回滚的模型能力。

D8 · 推理相关 7/25
4 NODES

推理服务架构

JD #12 #24 #25

Serving architecture

是什么 / WHY

理解 prefill/decode、连续批处理、排队、分布式推理、模型实例与请求路由。

传统锚点 / ANCHOR

高并发服务、GPU 集群、负载均衡

示例验收 / EVIDENCE

在代表性上下文和输出长度下建立吞吐、TTFT、TPOT、可用性和成本基线。

关键技术 / TECH

vLLM / SGLang / TensorRT-LLM / serving gateway

JD #12 #24 #25 · Sources vLLM Documentation ↗ SGLang Documentation ↗

KV Cache 与长上下文性能

JD #14 #21 #24 #25

KV cache & long context

是什么 / WHY

Agent 的长提示与多轮历史改变显存、缓存命中、抢占和延迟结构。

传统锚点 / ANCHOR

内存管理、缓存、性能分析

示例验收 / EVIDENCE

按共享前缀、上下文长度和并发分层测量缓存收益与质量/隔离影响。

关键技术 / TECH

PagedAttention / prefix cache / RadixAttention

JD #14 #21 #24 #25 · Sources vLLM Documentation ↗ SGLang Documentation ↗

量化、解码与质量联合调优

JD #12 #24 #25

Optimization & quality tradeoff

是什么 / WHY

量化、投机解码和批处理优化必须同时评估任务质量与尾延迟。

传统锚点 / ANCHOR

性能工程、数值精度、A/B

示例验收 / EVIDENCE

优化前后运行同一 Agent 任务集，报告速度、成本、质量和模型/引擎版本。

关键技术 / TECH

Quantization / speculative decoding / batching

JD #12 #24 #25 · Sources vLLM Documentation ↗ Anthropic · Demystifying evals for AI agents ↗

模型容量、路由与发布 Capacity, routing & release

JD #3 #12 #25

是什么 / WHY

管理模型版本、容量、租户配额、故障切换和跨引擎兼容；模型路由还需 B 的质量证据。

传统锚点 / ANCHOR

容量规划、灰度、服务网格

示例验收 / EVIDENCE

模型升级或故障切换时保持任务可恢复，并观察质量、延迟、成本和上下文兼容。

关键技术 / TECH

Model gateway / autoscaling / canary

JD #3 #12 #25 · Sources vLLM Documentation ↗

C2

后训练与 Agentic RL Post-training & Agentic RL

当 Harness 优化到达边界，轨迹、环境与奖励可进入模型能力改进，但需要独立的数据与安全体系。

D8 · 后训练 / RL 6/25
4 NODES

轨迹数据与血缘治理

JD #7 #15 #24

Trajectory data & lineage

是什么 / WHY

从任务轨迹构建训练数据，处理同意、隐私、许可、去重、污染、难例和失败样本。

传统锚点 / ANCHOR

数据工程、数据版本、标注

示例验收 / EVIDENCE

每个训练样本可追溯任务、环境、模型、Harness 和评分；训练/评测去泄漏。

关键技术 / TECH

Trajectory dataset / DVC / lineage / decontamination

JD #7 #15 #24 · Sources Anthropic · Demystifying evals for AI agents ↗

SFT 与偏好优化

JD #5 #7 #12 #15

SFT & preference optimization

是什么 / WHY

用高质量行为、工具使用与偏好数据调整模型；区分模仿表面轨迹与学习可泛化决策。

传统锚点 / ANCHOR

监督微调、偏好学习、数据筛选

示例验收 / EVIDENCE

在保留任务和真实环境评测泛化、回归与安全，而非只报告训练集指标。

关键技术 / TECH

SFT / DPO / reward modeling

JD #5 #7 #12 #15 · Sources Anthropic · Demystifying evals for AI agents ↗

RL 环境、奖励与探索

JD #7 #24

RL environments & rewards

是什么 / WHY

构建可复位环境、可验证奖励、工具交互和探索策略，处理奖励投机与部分可观察性。

传统锚点 / ANCHOR

RL、仿真环境、奖励设计

示例验收 / EVIDENCE

环境能检测代理奖励与真实 outcome 的偏离；训练配置、种子、版本和失败轨迹可复核。

关键技术 / TECH

Agentic RL / GRPO / verifiable rewards / simulator

JD #7 #24 · Sources Anthropic · Demystifying evals for AI agents ↗

模型-Harness 联合评测

JD #13 #14 #24

Model–harness co–evaluation

是什么 / WHY

模型与 Harness 是联合被测单元；模型升级可能改变工具使用、上下文需求和故障模式。

传统锚点 / ANCHOR

系统实验、协同优化、回归测试

示例验收 / EVIDENCE

在多个 Harness 与任务族上比较模型变更，识别协同收益、负迁移与安全回归。

关键技术 / TECH

Model × Harness matrix / online-offline eval

JD #13 #14 #24 · Sources Anthropic · Demystifying evals for AI agents ↗ Anthropic · Multi-agent research system ↗

C3

产品组合与组织领导 Strategy & organizational leadership

负责人需要交付方向、组织与经营系统，而不只是更复杂的架构图。

D9 · 负责人岗横切
4 NODES

机会组合与自治边界

JD #10 #12 #19 #20

Portfolio & autonomy strategy

是什么 / WHY

按业务价值、任务可判定性、风险与数据准备度排序机会，决定不做什么。

传统锚点 / ANCHOR

产品战略、风险管理、平台战略

示例验收 / EVIDENCE

形成有优先级、退出条件和非 Agent 替代方案的路线图，避免所有流程统一 Agent 化。

关键技术 / TECH

Opportunity scoring / risk tier / product portfolio

JD #10 #12 #19 #20 · Sources Anthropic · Demystifying evals for AI agents ↗

Build vs Buy 与投资回报

JD #10 #19 #20

Build/buy & ROI

是什么 / WHY

比较自研、框架、托管平台和模型提供方的全生命周期成本、锁定与差异化价值。

传统锚点 / ANCHOR

技术选型、FinOps、供应商管理

示例验收 / EVIDENCE

用同一业务纵切片评估交付、运维、质量、迁移和退出成本，而非只比较许可价格。

关键技术 / TECH

TCO / ROI / vendor exit / managed harness

JD #10 #19 #20 · Sources Amazon Bedrock AgentCore Harness GA ↗

团队拓扑与能力梯队

JD #12 #19 #20 #23

Team topology & talent

是什么 / WHY

定义平台、Agentic、模型系统、评测、领域和安全之间的责任与人才结构。

传统锚点 / ANCHOR

Conway 定律、平台团队、能力模型

示例验收 / EVIDENCE

关键制品有唯一 accountable owner，跨团队接口和升级路径明确；培训与实际交付绑定。

关键技术 / TECH

Platform team / embedded domain team / RACI

JD #12 #19 #20 #23 · Sources Anthropic · Demystifying evals for AI agents ↗

治理、采用与组织变更

JD #10 #19 #20 #23

Governance & adoption

是什么 / WHY

建立风险分级、发布责任、事故响应、用户培训、采用指标和政策更新机制。

传统锚点 / ANCHOR

技术治理、变更管理、运营机制

示例验收 / EVIDENCE

新 Agent 上线前有责任人和退出方案；上线后跟踪采用、接管、事故与业务价值并调整自治。

关键技术 / TECH

Operating model / governance / adoption metrics

JD #10 #19 #20 #23 · Sources OWASP AI Agent and MCP Security ↗

07 / MIGRATION & INTERVIEW

从既有优势进入，而不是从零开始

传统背景	优先节点	首个作品	必须重学的部分
后端 / 分布式	A2、A3、A4、B3	带持久化、幂等工具和恢复机制的 Agent 服务	非确定性、轨迹质量、模型/提示配置作为发布单元
SRE / K8s / 安全	A1、A2、A5、A6	可攻击、可观测、可销毁的沙箱执行平台	不受信内容会操纵控制流；可用性不等于任务成功
应用 / 全栈	B1、B5、B7、B8	有人工接管、领域评测和真实用户流程的 Agent	任务采样、语义评分、自治边界、失败体验
搜索 / 数据 / 推荐	B4、B6、C2	带权限、反馈和记忆纠正的知识型 Agent	记忆与检索不是同一层；轨迹数据会成为训练资产
ML Systems	C1、C2、A6、B6	模型 × Harness 的质量-延迟-成本联合评测	环境副作用、长任务恢复、工具语义、线上行为

技术负责人 / EM	C3、B6、B7、A5	有风险分级、ROI、退出机制和团队责任的服务组合	从功能交付转为概率系统经营；组织能力也是交付物

完整迁移矩阵：15 个传统能力 → Agent 对应物

保真度是旧经验可直接复用的程度，不是能力等级：高 可立即复用；中 需要重新设计边界；低 必须从零建立肌肉记忆。

传统 SWE 能力	Agent 时代的对应物	保真度	你必须重学的那部分
REST / RPC 服务设计	Tool / MCP Server	高	接口读者从人变成模型：description 与 schema 写含糊，就是一个必然发生的 bug。
OpenAPI 契约治理	Tool Schema 治理	高	版本与弃用要考虑模型可能缓存旧 schema；危险工具需要默认审批标记。
事务与幂等	Checkpoint + 幂等工具调用	中	状态是显式、可分支、可回放的；恢复靠事件重放而非回滚，副作用必须自己保证幂等。
工作流引擎 / MQ	Durable Agent Runtime	中	DAG 不能完全预编译——下一步由模型决定；引擎要支持运行中改变计划。
容器 / K8s 编排	microVM / 沙箱编排	高	威胁模型从「你写的可信代码」变成「别人写的敌意代码」；共享内核不再是默认假设。
缓存设计	Prompt / KV / 工具目录缓存	中	前缀稳定性直接决定成本；缓存断点放在哪是架构决策，不是配置项。
IAM / RBAC	Agent 身份 + OBO + 凭证代理	中	需要「代表用户行事但权限严格小于用户」；还要防「单次合法、序列非法」（读 + 外发）。
日志 / APM	OTel GenAI Trace + Transcript	高	span 里多了一层「意图与决策」语义；内容字段默认敏感，采集即合规决策。
单元测试	Eval Set + 轨迹 / 状态断言	中	断言对象从「输出」变成「路径与最终状态」；同一输入不保证同一输出，需重复试验。
性能优化	质量 / 时延 / 成本三维优化	低	三者互相拉扯；成本单位是 token、工具和执行时长，失败任务照样烧钱。
Code Review	Transcript / Trajectory Review	低	全新技能：读 Agent 的思考与试错轨迹，从「它试了 6 次才对」里判断是模型、工具还是环境问题。
灰度发布	灰度 + 模型版本回归	中	换模型等于换了一个不确定的上游依赖，行为漂移只能靠 eval 捕获，不能靠 diff。

多租户 SaaS	多租户 Agent 平台	高	新增两条租户边界：记忆的内存边界、凭证的注入边界。串一次就是事故。
编译期类型检查	结构化输出 + 校验 / 修复回合	中	校验发生在运行时；失败不是抛异常，而是要设计一个可观测、可停止的修复回合。
前端状态机	流式 UI + 可中断交互	中	用户可能中途改主意、断连、反悔；Agent 要能接住并保留现场，而不是从头再来。

高保真迁移与必须新增的直觉

可以直接复用

API/Schema 契约、幂等键、事件溯源、事务与补偿、IAM/OBO、限流配额、分布式追踪、缓存、灰度回滚、多租户、容量规划、SRE 事故响应。

必须新增

模型行为评估、间接提示注入与最小自治、上下文信息经济学、轨迹阅读、LLM Judge 校准、重复试验统计、模型-Harness 联合归因、每成功任务成本。

> **传统类比最容易误导的地方：**传统服务也会静默返回业务错误，Agent 也会 P99 超时；差异不是「旧系统确定、新系统随机」这么简单，而是模型现在参与决策、工具路径和上下文构造，导致质量、成本与副作用都呈路径依赖。

面试信号：把 JD 原话翻译成你要拿出的证据

面试官通常不会问「你懂不懂 Agent」，而是用一句 JD 原话追问具体经历。按「问题 → 证据 → 节点」准备故事，比背概念有效。

JD 里的原话（节选）	实际会问什么	你要拿出的证据	对应节点
「能基于 Trace、评测结果及用户反馈持续优化 Agent」	讲一次你的 Agent 为什么失败，你怎么定位的	一条具体 trace + 归因结论 + 修复后的回归数据	A6 · B6
「让『越用越好』从口号变成机制」	你的 badcase 怎么变成不重复犯	badcase → 规则/技能 → 回归用例 的闭环样例	B8 · B6
「保障长程 Agent 任务的稳定性与可恢复性」	进程挂了、断连了、跑了六小时，怎么办	检查点设计 + 幂等策略 + 恢复演练记录	A2
「围绕容器、虚拟机或轻量级 Sandbox 完善隔离」	为什么不用普通容器，逃逸你怎么防	选型三轴论证 + 一次红队验证结果	A1
「把业务约束转化为可执行规则、把任务状态转化为过程制品」	你怎么把一句模糊需求变成可执行的东西	一份规格或状态机设计，附落地效果	B2 · B7

「在信息保真与成本之间取得平衡」	上下文快满了你怎么处理，压掉了什么	压缩策略前后成功率、时延与成本对比	B3
「把模型、工具、存储等外部依赖抽象为插件，实现热插拔」	你怎么让换模型/换工具不改业务代码	一层抽象的设计与它的泄漏点	A3 · A4
「技能注册、版本管理、权限控制、调用契约」	一千个工具塞给模型，怎么不乱	检索式工具加载 + 权限围栏的设计	A3 · B5
「是 Agent 产品的高强度用户，对模型行为有品味」	你最近一次被 Agent 惊艳/气到是什么事	真实的、带细节的一手使用观察	B1 · B7
「构建 Agent 自进化、经验学习、自动评测」	你有没有让系统自己变好的机制	一个真正跑起来的自动化回路，哪怕很小	B8 · B6

08 / MATURITY & PORTFOLIO

用可复核证据定义成熟度

T1 · EXPLAIN

能画出一次任务的模型、工具、状态、身份、轨迹与副作用；知道 MCP、A2A、Skills、Runtime、Memory 和 Eval 各自解决什么。

T2 · BUILD

做出单 Agent 纵切片：有结构化工具、持久化、人工接管、基础追踪和任务集；不是只有聊天 UI。

T3 · OPERATE

能处理重试风暴、重复副作用、上下文膨胀、模型变更、注入攻击和成本异常；用实验和线上信号共同判断质量。

T4 · LEAD

定义跨团队接口、风险分级、投资优先级和组织责任；能证明为何采用或不采用多 Agent、自研平台或后训练。

12–16 周参考路线：每阶段产生可检查制品

W01–02

任务与风险基线

选一个真实、边界清晰的任务。定义成功、失败、风险等级、人工接管、任务分布和成本模型。制品：任务说明书 + 风险登记册 + 初版评测集。

W03–04

单 Agent 纵切片

完成模型-工具-结果循环，加入结构化输出、错误语义、trace 和最终状态校验。制品：可运行服务 + 失败轨迹库。

W05–07

持久化、隔离与恢复

实现幂等、检查点、超时/预算、沙箱、出口控制。分别演示崩溃恢复、记录回放与实时重跑，不混称确定性。制品：故障演练报告。

W08–10

上下文与质量实验

比较上下文/记忆策略、模型或 Harness 配置；固定版本，重复试验，报告均值、pass@1/pass^k（按需）、置信区间、成本和延迟。制品：实验报告 + 原始结果。

W11–12

生产信任与发布

加入 OBO、序列策略、租户隔离、红队、灰度与回滚。门槛由业务风险决定。制品：威胁模型 + 发布门禁 + 回滚演练。

W13–16

按方向专精

平台选 A1/A2；Agentic 选 B2/B4/B7；模型系统选 C1；研究选 C2；负责人选 C3。制品：一次有基线、有反证、有「不采用」结论的架构决策。

09 / SELECTION & ANTI-PATTERNS

框架选型：先确定控制需求，再看生态

选项示例	优先验证	不要只凭名称假设
LangGraph / LangChain	显式状态图、持久化、人工介入、语言生态	检查当前版本的执行与存储语义；图可控不等于自动获得幂等和隔离。
OpenAI Agents SDK	Agent loop、handoff、工具与追踪的集成方式	确认模型提供方、会话持久化和部署边界是否满足你的需求。
Claude Agent SDK	复用成熟 Harness、工具权限、编码/长任务场景	内置工具仍需隔离、出口、凭证和审计；不要把托管抽象当安全边界。
Google ADK / Microsoft Agent Framework	云生态、企业语言、工作流与部署集成	评估可移植性、版本迁移、既有 IAM/可观测体系的复用程度。
AgentScope / Spring AI	Java 与企业应用集成	不要只比较 API 数量；验证状态、恢复、评测与运维接口。
Dify 等可视化平台	快速产品化、非工程角色协作、托管运维	确认复杂控制流、私有协议、低层调试与数据治理是否可扩展。

统一选型评分：任务适配、状态与恢复、工具/协议、人工接管、可观测、评测、隔离与身份、模型可移植性、部署、团队语言、总拥有成本。用同一任务纵切片做 2–3 个候选的可重复比较。

八个仍然成立的反模式

反模式	修正
先上 Multi-Agent，再找理由	先证明任务有可并行、可隔离或超出单上下文的部分；同时测量协调成本和失败传播。
把 Prompt 当普通文本配置	将模型、Prompt、工具 Schema、Skills、检索配置和 Harness 一起版本化并评测。
排行榜直接决定生产选型	先比较任务相似度、污染、评分器、Harness、成本和日期；最终使用自有任务集。
有容器就有安全边界	从威胁模型选择隔离、出口、凭证和策略；对高风险不受信代码单独评估 microVM/VM。
有 Checkpoint 就能确定性回放	区分状态恢复、记录回放和实时重跑；记录模型/工具 I/O 并处理外部副作用。
LLM Judge 上线即可信	用人工标注校准，报告一致性和偏差，定期抽检并防止被评分器投机。
只压每次调用 token 单价	优化每成功任务总成本：模型、重试、工具、沙箱、存储、网络、人工复核与失败损失。
把「不采用」当技术失败	有实验依据地不采用多 Agent、长期记忆、自研框架或后训练，往往是最成熟的架构结论。

10 / TECHNOLOGY REGISTER

技术登记册：叶脉层，可整体替换

能力节点回答「要会什么」，登记册回答「现在用什么」。技术项不是能力的定义，而是当前可用的实现；它们每 6–18 个月就会换一批，所以单独放一页，标注类型与状态，而不是写进能力树里。

> **怎么用这一页：**先用节点编号（A1、B6…）定位能力，再看这一行有哪些实现可选，最后用状态列判断它还能信多久。S0 稳定 表示接口与语义已收敛；S1 演进 表示仍在快速变化，必须固定版本；S2 易变 表示规范或生态近期有结构性变更。

规范 规范与标准　框架 开发框架　运行时 基础设施　托管 托管服务　基准 基准与评测　实践 实践与模式

技术项	类型	域	节点	状态	在能力链上的作用	一手入口
OWASP AI Agent & MCP Security	实践	D0	A5	S0	最小自治、间接注入与工具供应链的基线清单	OWASP AI Agent and MCP Security ↗
OPA / Rego	运行时	D0	A5	S0	策略即代码；序列策略需额外上下文	Open Policy Agent ↗
OpenFGA	运行时	D0	A5	S1	对象级与租户级关系授权	OpenFGA Documentation ↗
OAuth 2.1 + On-Behalf-Of	规范	D0	A5	S0	代表用户行事且权限严格小于用户	—
SPIFFE / SPIRE	规范	D0	A5	S0	工作负载身份，替代长期静态密钥	—

技术项	类型	域	节点	状态	在能力链上的作用	一手入口
审计事件模型	实践	D0	A5	S1	谁在何时以何种身份做了什么，可回溯	—
Firecracker	运行时	D1	A1	S0	microVM 边界；启动与开销须在自有负载测量	Firecracker ↗
gVisor	运行时	D1	A1	S0	用户态内核；拦截系统调用，兼容性有代价	gVisor ↗
Kata Containers	运行时	D1	A1	S0	容器接口 + VM 隔离的组合	Kata Containers (GitHub) ↗
seccomp / Landlock	运行时	D1	A1	S0	系统调用与文件访问收敛，非完整隔离	—
Playwright	运行时	D1	A1	S0	浏览器自动化与可复现执行环境	Playwright Documentation ↗
Temporal	运行时	D1	A2	S0	持久化执行、重试、定时器与恢复	Temporal Documentation ↗
Kubernetes 调度与配额	运行时	D1	A2	S0	优先级、抢占、配额与公平性	Kubernetes Scheduling ↗
幂等键 + Outbox/Inbox	实践	D1	A2	S0	让重试不产生重复副作用	—
Saga / 补偿事务	实践	D1	A2	S0	跨系统失败后的业务一致性	—
运行预算与熔断	实践	D1	A2	S0	步数、时长、token 与费用的可取消上限	—
MCP 2026-07-28	规范	D2	A3	S2	无状态核心、header 路由、MRTR、Tasks 扩展	MCP 2026-07-28 release ↗
A2A v1.0	规范	D2	A3	S1	跨 Agent 发现、任务与签名的 Agent Card	A2A v1.0.0 Specification ↗
Agent Skills / SKILL.md	规范	D2	B5	S1	渐进披露的能力包格式	Agent Skills Specification ↗
JSON Schema	规范	D2	A3	S0	工具参数与输出的机器契约	—
OpenAPI 3.1	规范	D2	A3	S0	既有 HTTP 资产转工具契约的起点	—
工具描述评测集	实践	D2	B5	S1	用模型选择准确率检验描述质量	Anthropic · Demystifying evals for AI agents ↗
SBOM 与签名制品	实践	D2	A3	S0	Skill 与 MCP server 的供应链审查	—
协议网关与认证	运行时	D2	A3	S0	把鉴权、限流、审计移出模型	—
前缀缓存	运行时	D3	B3	S1	稳定前缀决定成本与延迟	Anthropic · Effective context engineering ↗
压缩与摘要	实践	D3	B3	S1	阈值按模型与任务测量，不用固定百分比	Anthropic · Effective context engineering ↗
子 Agent 上下文隔离	实践	D3	B3	S1	用委派换取上下文纯度	Anthropic · Multi-agent research system ↗

技术项	类型	域	节点	状态	在能力链上的作用	一手入口
向量检索 HNSW/IVF	运行时	D3	B4	S0	近似最近邻的召回与延迟权衡	—
混合检索 + 重排	实践	D3	B4	S0	稀疏 + 稠密 + rerank 的组合基线	—
结构化记忆	实践	D3	B4	S1	情节与语义记忆的写入、召回与冲突	—
记忆纠正与遗忘	实践	D3	B4	S1	用户可纠正、可删除、可审计	—
文档级 ACL 检索	实践	D3	B4	S0	检索结果不得越权，过滤发生在召回前	—
ReAct 循环	实践	D4	B1	S0	观察-思考-行动的最小闭环	—
LangGraph	框架	D4	B2	S1	显式状态图、持久化与人工介入	LangGraph Documentation ↗
OpenAI Agents SDK	框架	D4	B1	S1	Agent loop、handoff 与追踪	OpenAI Agents SDK ↗
Claude Agent SDK	框架	D4	B1	S1	成熟 Harness 与工具权限模型	Claude Agent SDK ↗
Google ADK	框架	D4	B1	S1	云生态、语言与部署集成	Google Agent Development Kit ↗
Microsoft Agent Framework	框架	D4	B1	S1	企业语言与工作流集成	Microsoft Agent Framework ↗
AgentScope	框架	D4	B2	S1	多 Agent 与 Java 生态	AgentScope Documentation ↗
Dify	托管	D4	B1	S1	可视化编排与托管运维	Dify Documentation ↗
编排器-工作者模式	实践	D4	B2	S1	并行与专门化，代价是协调与失败传播	Anthropic · Multi-agent research system ↗
停止条件与循环检测	实践	D4	B2	S0	终止原因必须可观测，而非只设最大步数	—
人工接管与审批门	实践	D4	B2	S0	关键动作默认需要确认与可中断	—
任务集与留出集	实践	D5	B6	S0	任务分布比任务数量更重要	Anthropic · Demystifying evals for AI agents ↗
pass@k / pass^k	实践	D5	B6	S0	能力上限与重复一致性是两个问题	Anthropic · Demystifying evals for AI agents ↗
LLM-as-judge + 人工校准	实践	D5	B6	S1	报告一致性、偏差并定期抽检	Anthropic · Demystifying evals for AI agents ↗
轨迹评分与结果评分	实践	D5	B6	S0	路径对不等于结果对，两者分开评	Anthropic · Demystifying evals for AI agents ↗
SWE-bench	基准	D5	B6	S1	仓库级修复；注意污染与 Harness 口径	SWE-bench ↗
Terminal-Bench	基准	D5	B6	S1	终端长任务的评测入口	Terminal-Bench ↗

技术项	类型	域	节点	状态	在能力链上的作用	一手入口
OSWorld	基准	D5	B6	S1	计算机使用的最终状态评测	OSWorld ↗
AgentBench / GAIA	基准	D5	B6	S1	通用 Agent 能力的历史基线	—
CI 评测门禁	实践	D5	A7	S1	把回归变成合并前条件，而非发布后检查	—
影子流量对比	实践	D5	A7	S1	新旧配置在同分布上对照，不只看离线集	—
OTel GenAI 语义约定	规范	D6	A6	S2	invoke_agent / execute_tool 的轨迹模型	OpenTelemetry GenAI Observability ↗
semantic-conventions-genai	运行时	D6	A6	S2	仍在 Development，必须固定版本	OTel semantic-conventions-genai ↗
Trace + Transcript 双记录	实践	D6	A6	S1	结构化 span 与可读轨迹互补	—
成本归集模型	实践	D6	A6	S1	token、工具、沙箱与人工分别记账	—
每成功任务成本	实践	D6	A6	S1	失败任务同样计费，不能只算成功调用	—
内容脱敏与保留策略	实践	D6	A6	S0	采集内容本身就是合规决策	—
Amazon Bedrock AgentCore	托管	D7	A4	S1	托管 Harness 与平台原语的参照系	Amazon Bedrock AgentCore Harness GA ↗
Agent 定义清单	实践	D7	A4	S1	模型、工具、权限与预算一起版本化	—
GitOps 与不可变版本	实践	D7	A4	S0	变更可追溯、可回滚、可重放	—
Backstage / IDP	运行时	D7	A4	S0	模板、目录与自助发布	—
测试替身与环境虚拟化	实践	D7	A4	S1	模型、工具、用户与时间的替身	—
多租户命名空间与配额	实践	D7	A4	S0	隔离与公平性的第一道边界	Kubernetes Scheduling ↗
vLLM	运行时	D8	C1	S0	连续批处理、PagedAttention 与量化	vLLM Documentation ↗
SGLang	运行时	D8	C1	S1	调度与缓存优化的推理运行时	SGLang Documentation ↗
连续批处理	运行时	D8	C1	S0	吞吐与尾延迟的核心权衡	vLLM Documentation ↗
KV cache 与分页	运行时	D8	C1	S0	长上下文直接转化为显存成本	vLLM Documentation ↗
量化 FP8 / INT4	实践	D8	C1	S1	质量与成本的显式交换，需要回归验证	—
推测解码	实践	D8	C1	S1	用小模型换延迟，收益依任务而定	—
TTFT / TPOT / 吞吐	实践	D8	C1	S0	三类指标不能互相替代	—
SFT / DPO	实践	D8	C2	S0	行为模仿与偏好对齐	—

技术项	类型	域	节点	状态	在能力链上的作用	一手入口
GRPO / RLVR	实践	D8	C2	S1	可验证奖励驱动的策略改进	—
轨迹数据与奖励环境	实践	D8	C2	S1	环境与奖励定义决定能力上限	—
污染控制与数据血缘	实践	D8	C2	S0	训练集与评测集必须可隔离、可追溯	—
人机协作模式	实践	D9	B7	S0	建议、确认、代执行三档，按后果选择	—
流式与可中断交互	实践	D9	B7	S0	用户中途改主意要能接住并保留现场	—
多模态输入输出	实践	D9	B7	S1	语音、视觉与文档的边界处理	—
badcase → 回归用例	实践	D9	B8	S1	把一次失败变成永久约束	Anthropic · Demystifying evals for AI agents ↗
AI-native SDLC	实践	D9	B8	S1	规格、评审与测试随生成式开发重构	—
风险分级与自治边界	实践	D9	C3	S1	按后果而非按技术选自治等级	NIST AI Risk Management Framework ↗
ROI / TCO / 退出成本	实践	D9	C3	S0	包含迁移、运维与供应商锁定	—
RACI 与团队拓扑	实践	D9	C3	S0	关键制品必须有唯一 accountable owner	—
ISO/IEC 42001	规范	D9	C3	S0	AI 管理体系与生命周期要求	ISO/IEC 42001 ↗
SFIA 9 责任等级	规范	D9	C3	S0	用自主性与影响定义等级，而非职级名称	SFIA 9 ↗

登记册的维护规则

规则	说明
只登记被节点引用的技术	登记册不是「所有 AI 工具清单」。一条技术项必须能回答「它支撑哪个能力节点」；答不上来的不收录。
状态必须可辩护	S0/S1/S2 依据是规范状态或官方文档的稳定性表述，不是主观感觉。规范进入 Development 阶段即不得标 S0。
新增不影响能力树	新增或替换技术项只改这一页；只有当一项技术改变了**能力边界**（例如出现新的隔离原语）时，才修改能力节点。
链接优先一手	优先规范、官方文档、官方仓库。没有一手入口的条目留空，不用二手文章填充。

11 / SOURCES & REPRODUCIBILITY

来源登记：一条事实对应一类证据

所有链接均为规范、官方文档、官方工程文章或官方代码/基准仓库。这里不使用二手文章支撑协议状态、产品功能或性能常数。访问日期：2026-10-28。

> **文档约定：**全图只有一份能力数据（9 个域 + 1 个横切带 / 18 个分支 / 75 个能力节点），用六个视图投影：分层、领域森林、生命周期、控制面、自治等级、角色；Harness 为横切镜头。技术项单独登记（86 条，标注类型与状态）。每个能力节点都附 JD 编号与一手技术入口，另有 15 行迁移矩阵与面试信号表。

ID	来源	类型	用于支撑
a2a	A2A v1.0.0 Specification ↗	官方规范	Agent Card、JWS/JCS 签名、任务与认证、安全边界
aaif	Linux Foundation · Agentic AI Foundation ↗	官方公告	MCP、goose、AGENTS.md 作为创始项目进入中立治理（2025-12-09）
adk	Google Agent Development Kit ↗	官方文档	ADK 能力、语言与部署选型入口
agentops	AWS · AgentOps on AgentCore ↗	官方工程	AgentOps 阶段与平台原语；生命周期与控制面视图
agents	Anthropic · Building effective agents ↗	官方工程	工作流与 Agent 的边界；自治程度按任务选择
agentscope	AgentScope Documentation ↗	官方文档	Java/多 Agent 生态选型入口
aws	Amazon Bedrock AgentCore Harness GA ↗	官方文档	托管 Harness 的边界：循环、工具、状态、隔离、身份与可观测
claude	Claude Agent SDK ↗	官方文档	成熟 Harness、工具与权限模型的框架入口
cncf	CNCF 平台工程术语表 ↗	官方术语	平台能力与可替换 provider 的区分；能力/技术项分离的依据
context	Anthropic · Effective context engineering ↗	官方工程	上下文是有限资源；压缩与检索需按任务测量
dify	Dify Documentation ↗	官方文档	可视化 Agent/工作流平台选型入口
etclovg	ETCLOVG Harness survey (OpenReview) ↗	学术投稿（已撤回）	仅作七层词汇参考；稿件已撤回，不作为结构依据
eval	Anthropic · Demystifying evals for AI agents ↗	官方工程	任务、trial、grader、outcome、pass@k/pass^k 与评测集维护
firecracker	Firecracker ↗	官方文档	microVM 隔离模型；性能必须在自有工作负载测量
gvisor	gVisor ↗	官方文档	用户态内核、系统调用拦截与兼容性/开销权衡

iso	ISO/IEC 42001 ↗	国际标准	AI 管理体系与生命周期要求；治理阶段与退出条件
k8s	Kubernetes Scheduling ↗	官方文档	优先级、抢占、配额与调度语义
kata	Kata Containers (GitHub) ↗	官方仓库	容器接口与 VM 隔离组合；文档站 404，改用官方仓库
langgraph	LangGraph Documentation ↗	官方文档	显式状态图、持久化与人工介入的选型入口
mcp	MCP 2026-07-28 release ↗	官方规范	无状态核心、header routing、MRTR、Tasks 扩展、CIMD/DCR 变化
msagent	Microsoft Agent Framework ↗	官方文档	微软生态的 Agent 与工作流入口
multi	Anthropic · Multi-agent research system ↗	官方工程	多 Agent 适用条件、协调成本；4x/15x 只属于其特定工作负载
nice	NICE Framework (CISA) ↗	技能框架	工作角色与任务/能力分离；角色只作投影
nist	NIST AI Risk Management Framework ↗	官方框架	GOVERN/MAP/MEASURE/MANAGE 四功能；治理为横切功能
openai	OpenAI Agents SDK ↗	官方文档	Agent loop、handoff、工具与追踪的框架入口
openfga	OpenFGA Documentation ↗	官方文档	关系授权与租户/对象级权限锚点
opf	Open Policy Agent ↗	官方文档	策略即代码；Agent 序列策略需要额外上下文与状态
osworld	OSWorld ↗	官方基准	计算机使用环境、最终状态与任务评测入口
otel	OpenTelemetry GenAI Observability ↗	官方文档	invoke_agent、chat、execute_tool 轨迹结构与内容采集的隐私默认
otelrepo	OTel semantic-conventions-genai ↗	官方仓库	GenAI/MCP 语义约定仍在 Development，需固定版本
owasp	OWASP AI Agent and MCP Security ↗	官方指南	最小自治、间接注入、凭证、出口、工具供应链与沙箱
playwright	Playwright Documentation ↗	官方文档	浏览器自动化与可复现执行环境的工程入口
sfia	SFIA 9 ↗	技能框架	七级责任由通用属性（自主性、影响、复杂度、知识）定义；本图压缩为四档
sglang	SGLang Documentation ↗	官方文档	推理运行时、调度与缓存机制的选型入口
skills	Agent Skills Specification ↗	官方规范	SKILL.md、渐进披露、约 100 token 元数据与正文 <5000 token 建议

swebench	SWE-bench ↗	官方基准	基准版本、任务与 Harness 口径；本页不引用易变头部分数
temporal	Temporal Documentation ↗	官方文档	持久化执行、重试、定时器与恢复的传统工程锚点
terminal	Terminal-Bench ↗	官方基准	终端任务评测入口；比较时固定 Agent、模型和配置
vllm	vLLM Documentation ↗	官方文档	连续批处理、KV cache、量化与推理服务调优

JD 方法的可复现边界

> **运行：** `python3 jd-capability-evidence.py 目标JD标准库-Agent平台与负责人岗.md --pretty`
> 脚本包含完整正则、岗位编号、核心视图排除项和限制说明。语料 SHA-256 为 `dac3b44268f2eab14bf2561c5399392b21cf8f0128abee9adfb6efc925738f79`。若 JD 文件改变，图表必须重新生成。

> **仍然存在的限制：** 25 份 JD 不是行业随机样本；中文大模型公司占比高；正则词族不能替代逐条语义编码；能力节点中的示例验收是教学设计，不是行业合规标准；官方规范也会继续演进。
